Adversarial Search

Kaiqi Zhao
Harbin Institute of Technology, Shenzhen

Search v.s. Games

- Search – no adversary
  - Solution is (heuristic) method for finding goal
  - Aim to find *optimal* solution
  - Evaluation function: estimate of cost from start to goal through given node
  - Examples: path planning, scheduling activities
- Games – adversary
  - Solution is strategy (strategy specifies move for every possible opponent reply).
  - Time limits force an *approximate* solution
  - Evaluation function: evaluate "goodness" of game position
  - Examples: chess, checkers, Othello, backgammon

Types of Games

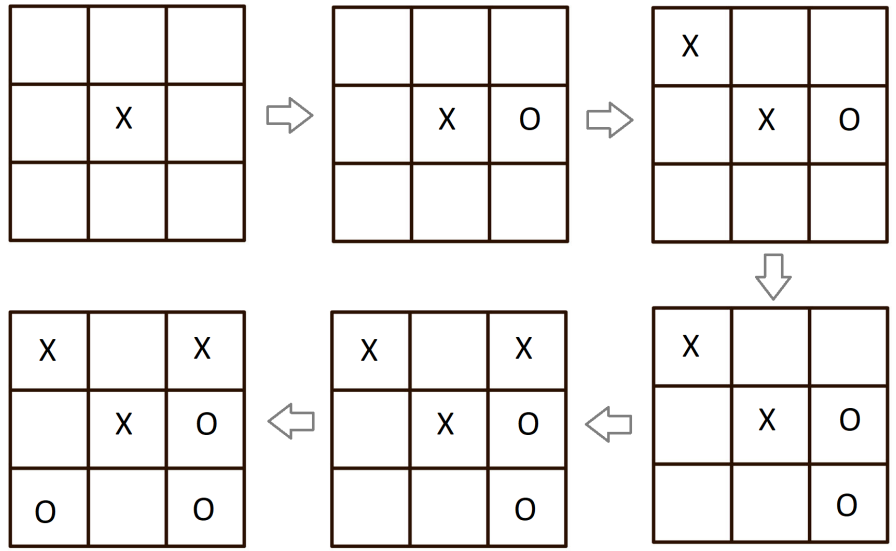
- Many different kinds of games!

- Axes:
  - Deterministic or stochastic?
  - One, two, or more players?
  - Zero sum?
  - Perfect information (can you see the state)?

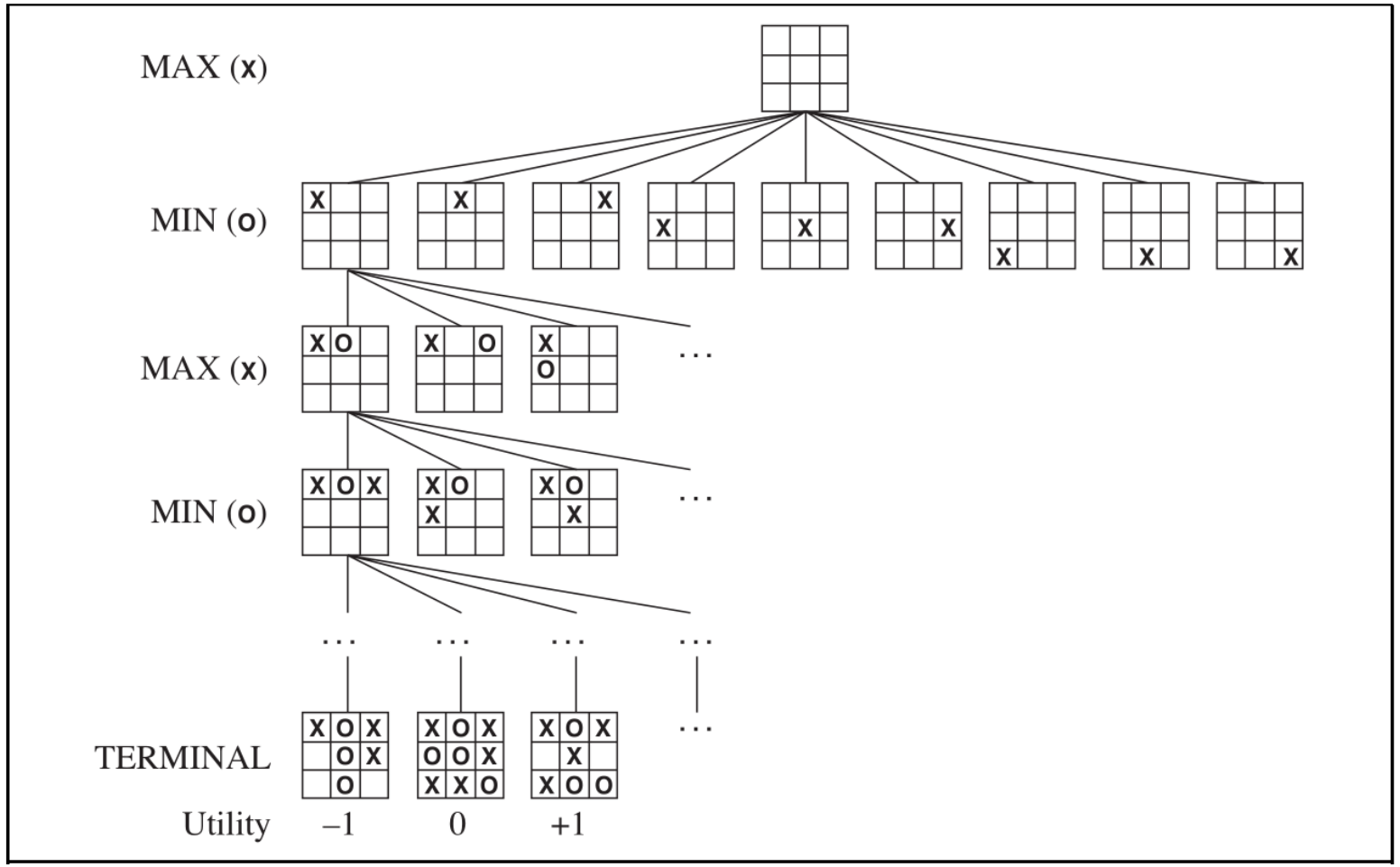
- Want algorithms for calculating a strategy (policy) which recommends a move from each state

Example. [**tic-tac-toe**]

- Two players, putting “O” or “X” on the board
- The two players alternative in moves.
- Every play of the game generate a state.
- A winner is declared when the game ends.



The tic-tac-toe game can be represented by a game tree



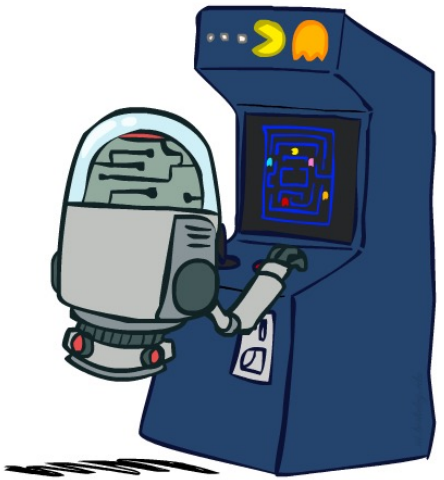
Suppose our agent act first (X), we get positive payoff + 1 when we win, otherwise negative payoff -1

Deterministic Games

Definition

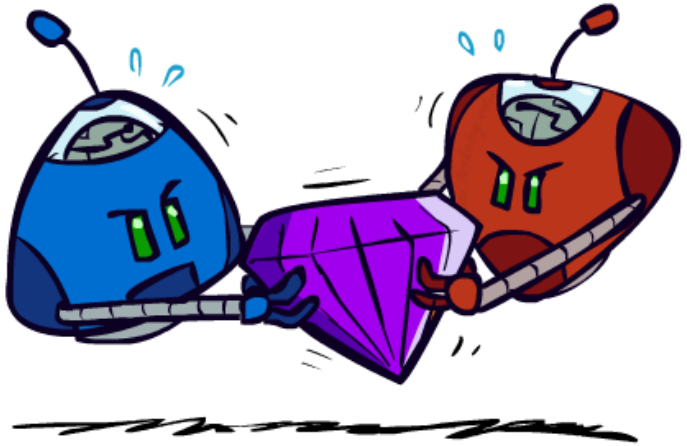
A deterministic game is defined as follows:

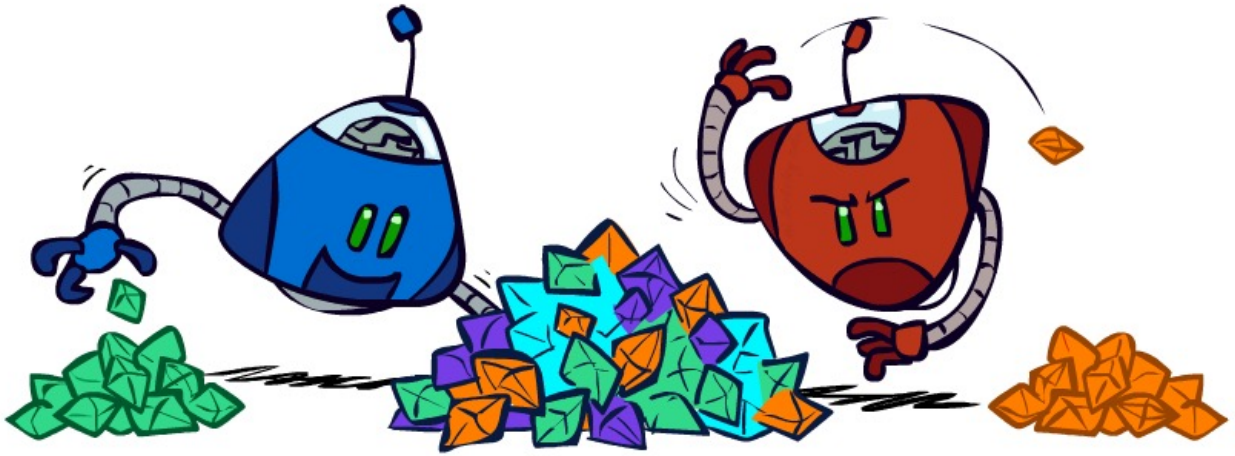
- States: S (start at s_0, end at any states in $F \subseteq S$)
- Players: $P = \{1 \ldots N\}$ (take turns)
- Actions: A
- Transition Function $R(s, a)$: $S \times A \rightarrow S$
- Payoff(s, i): $S \times P \rightarrow R$, payoff to player i in terminal state
- Player(s)$\in P$: The player who moves at state s
- Action(s)$\in A$: Actions available in state s



A zero-sum game is a game in which $\sum_{i \in N} \text{Payoff}(s, i) = 0$ for every state $s \in F$

Zero-Sum Games



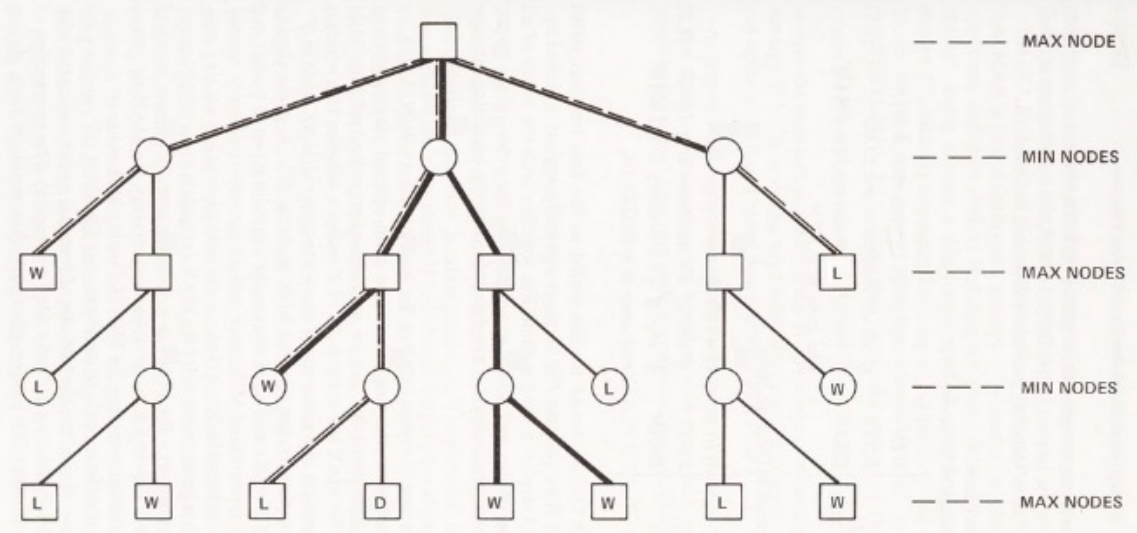


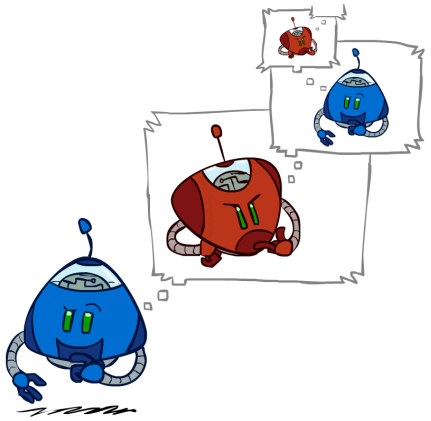
- **Zero-Sum Games**
  - Agents have opposite utilities (values on outcomes)
  - Lets us think of a single value that one maximizes and the other minimizes
  - Adversarial, pure competition
- **General Games**
  - Agents have independent utilities (values on outcomes)
  - Cooperation, indifference, competition, and more are all possible
  - More later on non-zero-sum games

Definition

A game tree is a representation of possible plays of the game:

- Every node is labeled by a state $s \in S$, and Player(s)
- The root of the tree is labeled by s_0
- Each edge is labeled by an action $a \in A$
- If a node has label s and $R(s, a) = s'$, then the node has a child with label s', and the edge is labeled by a
- The leaves are labeled by terminal states, and the payoff of players in that state.



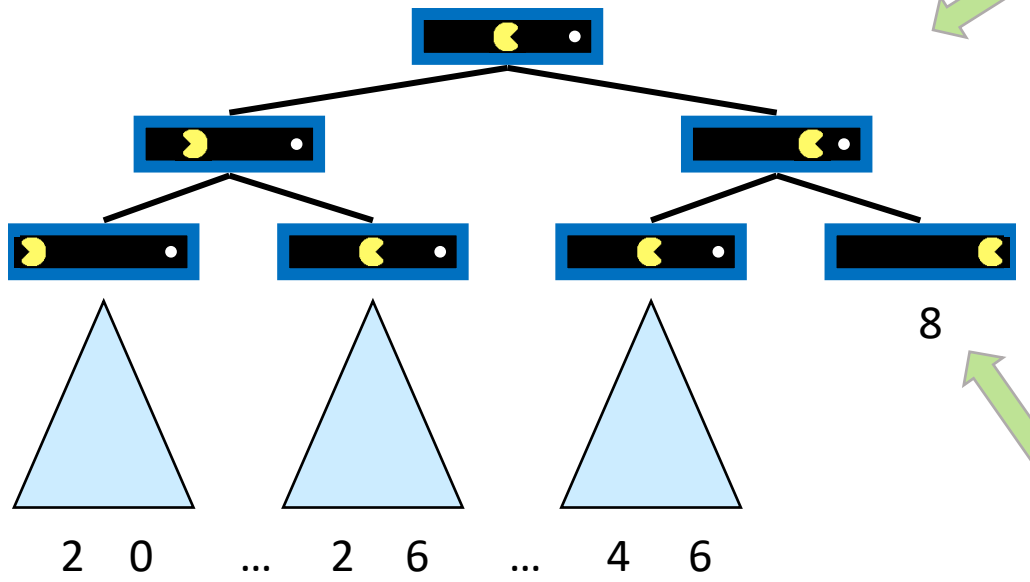


Value of a State

Value of a state: The best achievable payoff (or utility) from that state

Non-Terminal States:

$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$



Terminal States:

$$V(s) = \text{known}$$

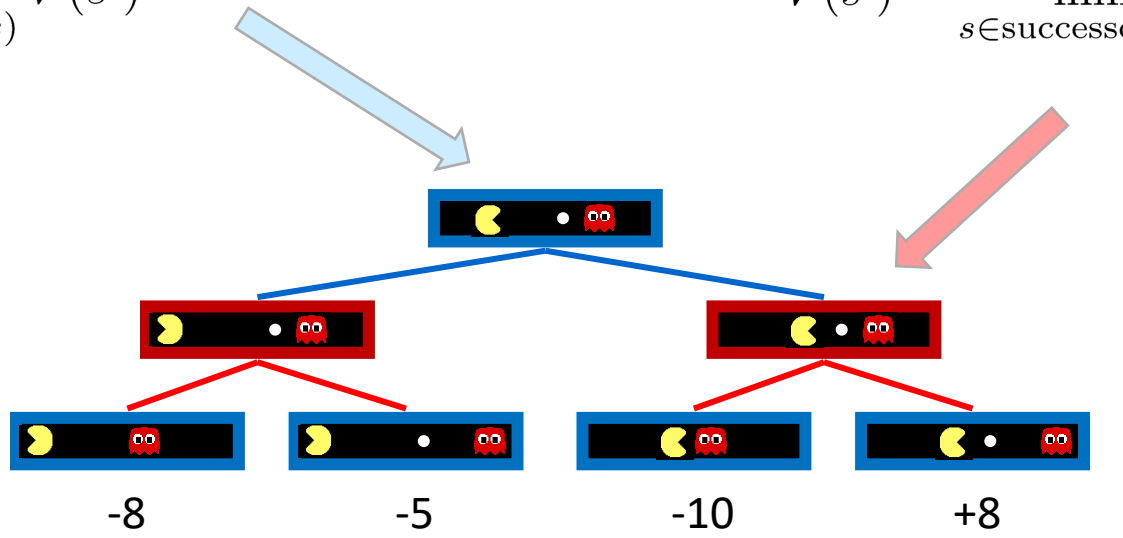
Minimax Values

States Under Agent's Control:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

States Under Opponent's Control:

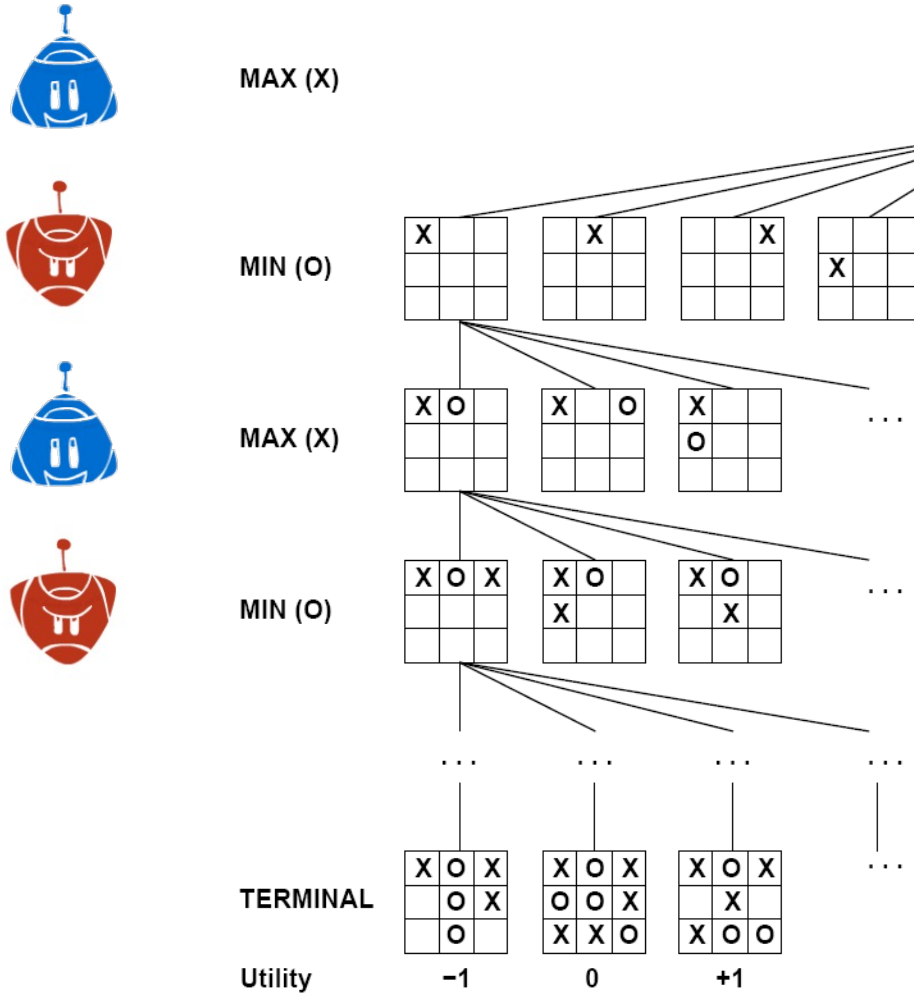
$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

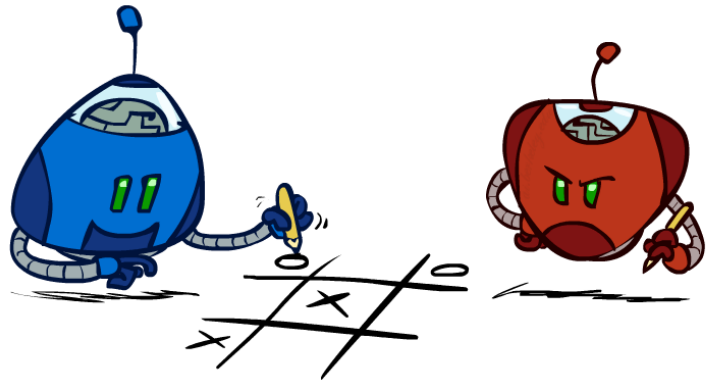


Terminal States:

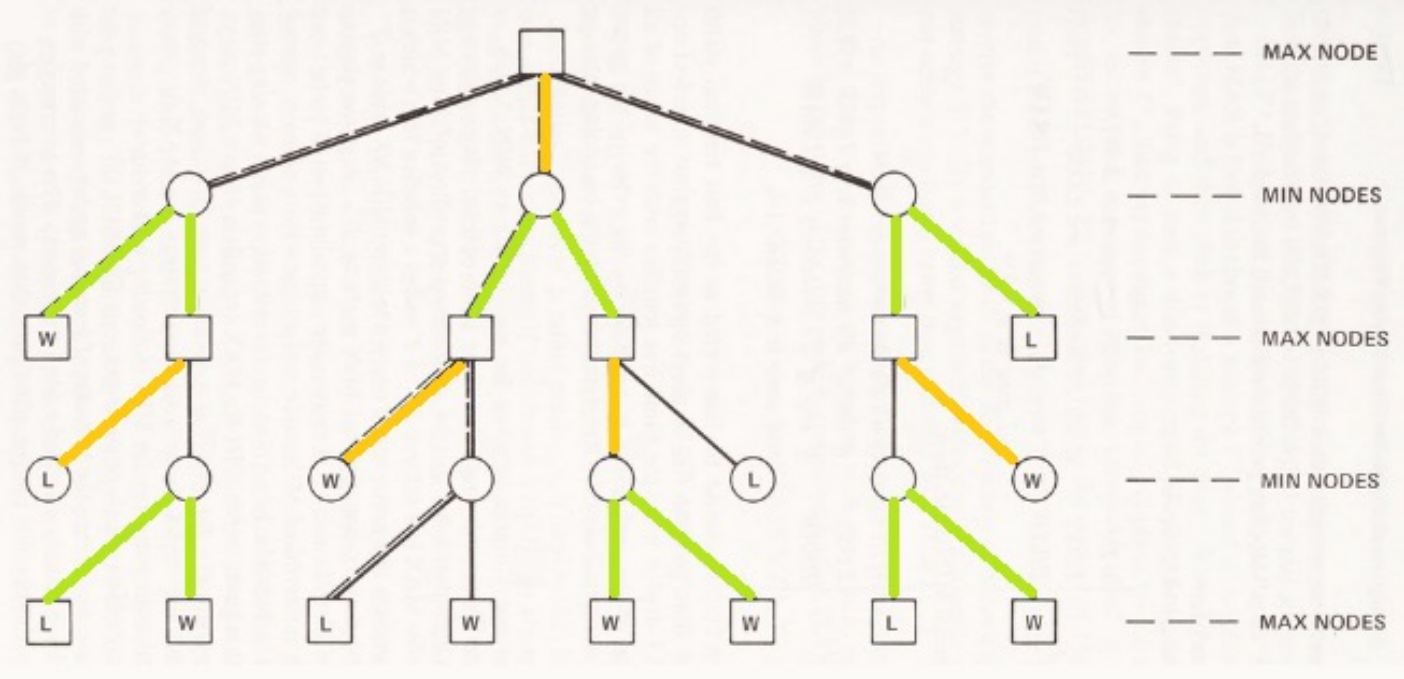
$$V(s) = \text{known}$$

Tic-Tac-Toe Game Tree





- In a game, instead of looking for a sequence of actions, an agent needs a contingency plan.
- Each player i takes some policy (or strategy) to decide which action to choose at a specific state.
- An optimal policy is one that leads to outcomes no worse than any other strategy when a player i is playing an infallible opponent.



Adversarial Search

Consider two-player zero-sum games. The two players are MAX and MIN:

- Payoff(s, MAX) + Payoff(s, MIN) = 0
- Let's focus on the agent for player MAX. By default, we use Payoff(s) to denote Payoff(s, MAX).

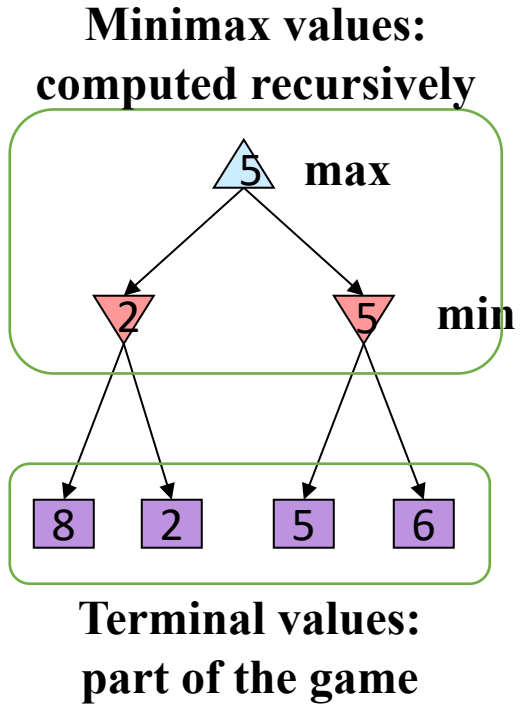
The adversarial search process is defined on a game tree such that:

- The search is carried out by two players having opposing goals:
  - MAX wants to maximise payoff(s)
  - MIN wants to minimise payoff(s)
- The two players take turn to select a successor.
- For any player, the goal is to find an optimal strategy for the game that starts from the root of the game tree.

Adversarial Search - Minimax

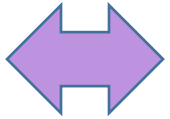
- Deterministic, zero-sum games:
  - Tic-tac-toe, chess, checkers
  - One player maximizes result
  - The other minimizes result

- Minimax search:
  - A state-space search tree
  - Players alternate turns
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary



Minimax Implementation

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```



```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

Minimax Algorithm

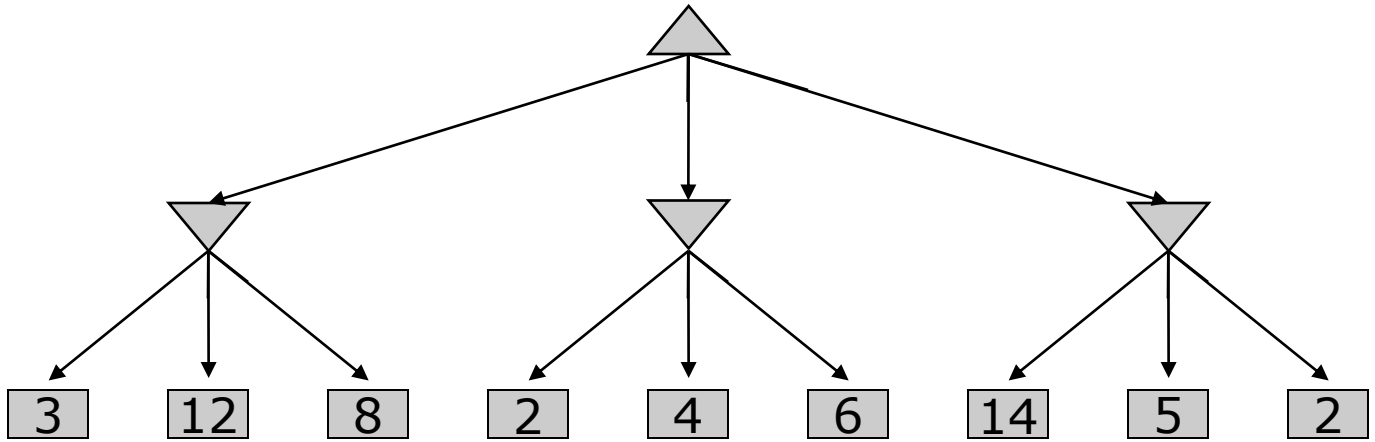
```
function MINIMAX-DECISION(state) returns an action
    v ← MAX-VALUE(state)
    return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s))
    return v

function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s))
    return v
```

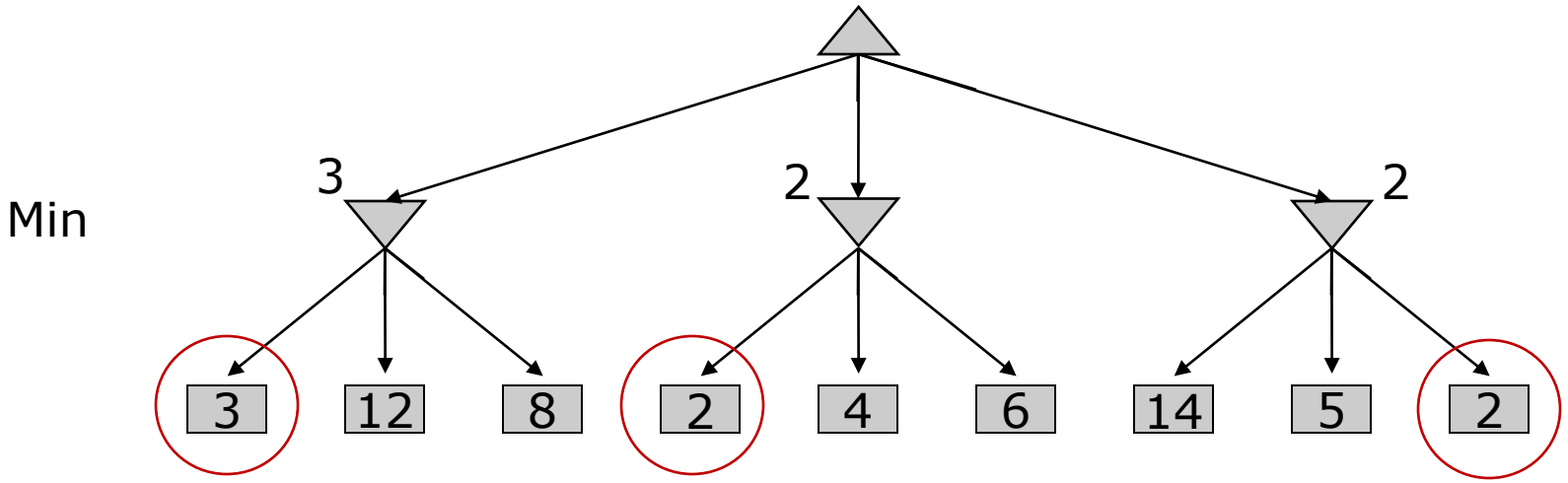
Minimax Example

Two-Ply Game Tree



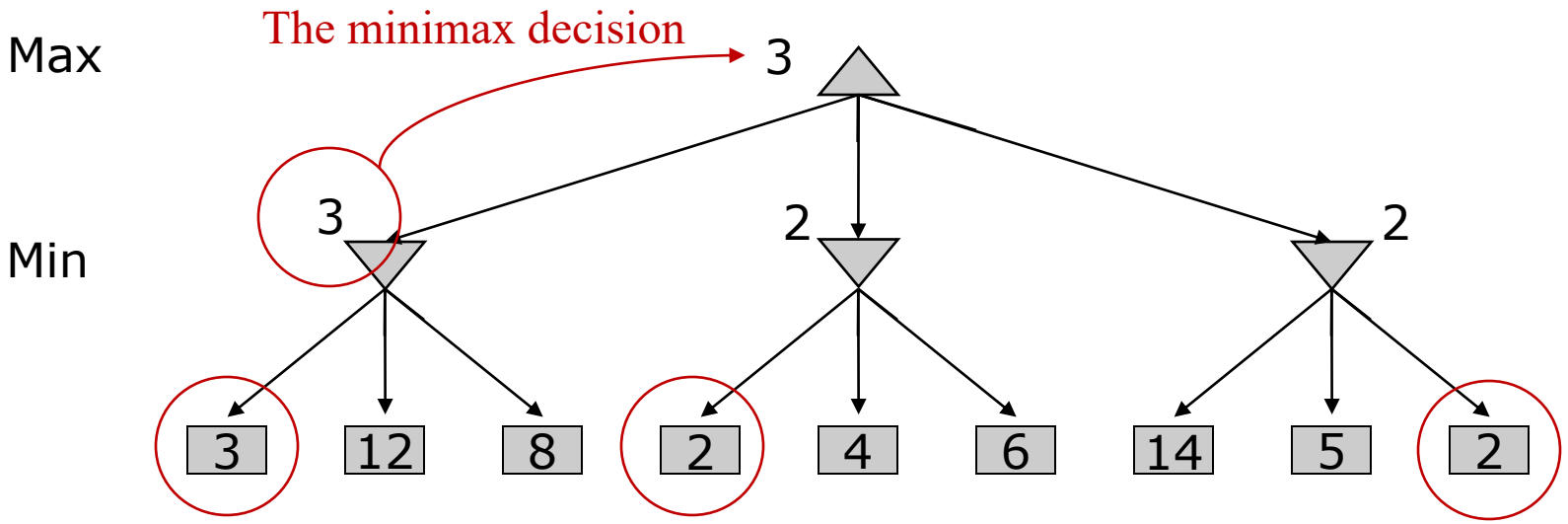
Minimax Example

Two-Ply Game Tree



Minimax Example

Two-Ply Game Tree

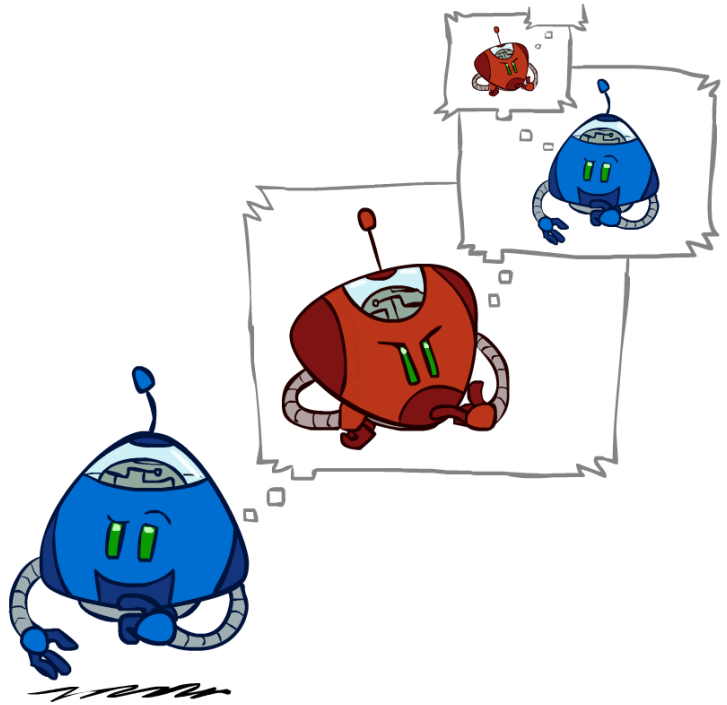


Minimax Efficiency

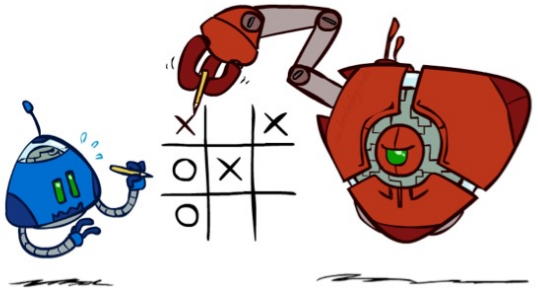
- How efficient is minimax?
  - Just like (exhaustive) DFS
  - Time: $O(b^m)$
  - Space: $O(bm)$

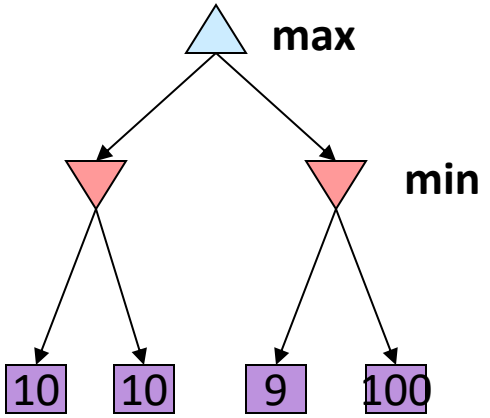
- Example: For chess, b ≈ 35, m ≈ 100
  - Exact solution is completely infeasible
  - But, do we need to explore the whole tree?

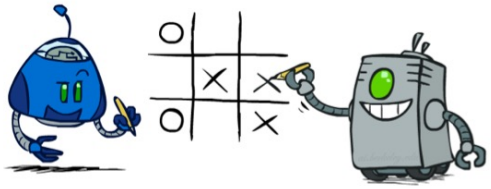
  - $35^{100} \approx 10^{154}$
  - The number of atoms in the universe is between 10^{78} and 10^{82}



Minimax Properties

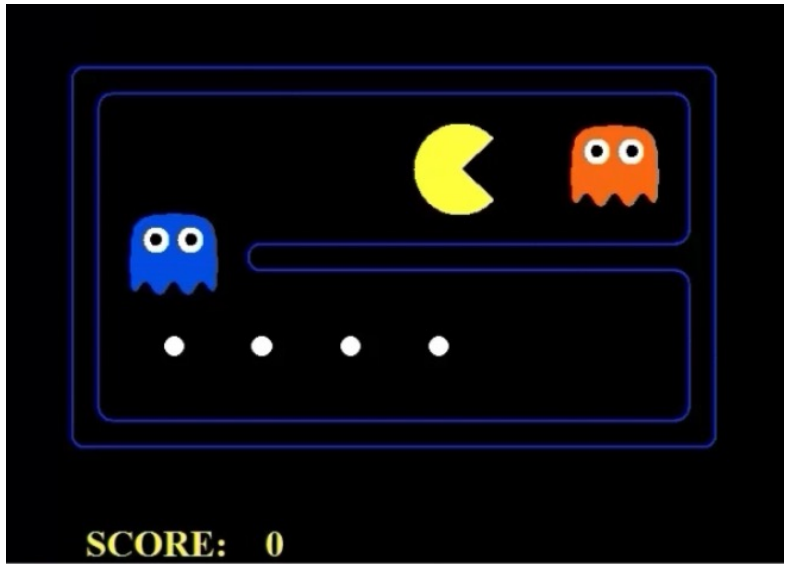






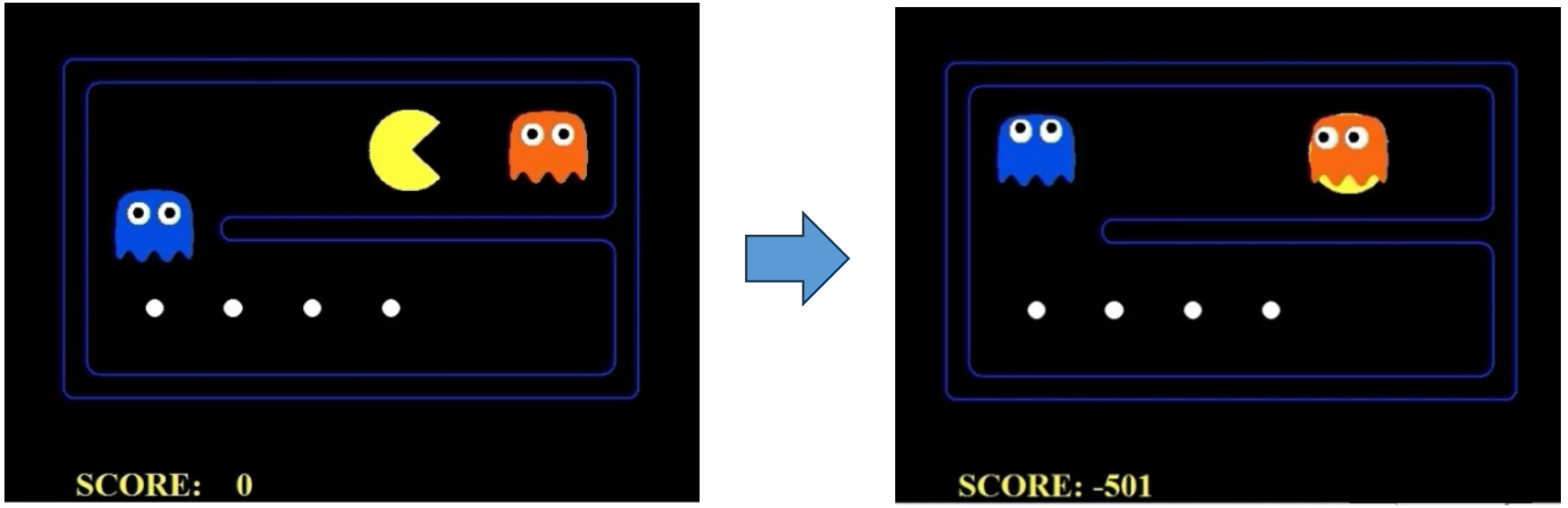
Optimality: Minimax is optimal against a perfect player. Otherwise?

Perfect v.s. Random Opponent



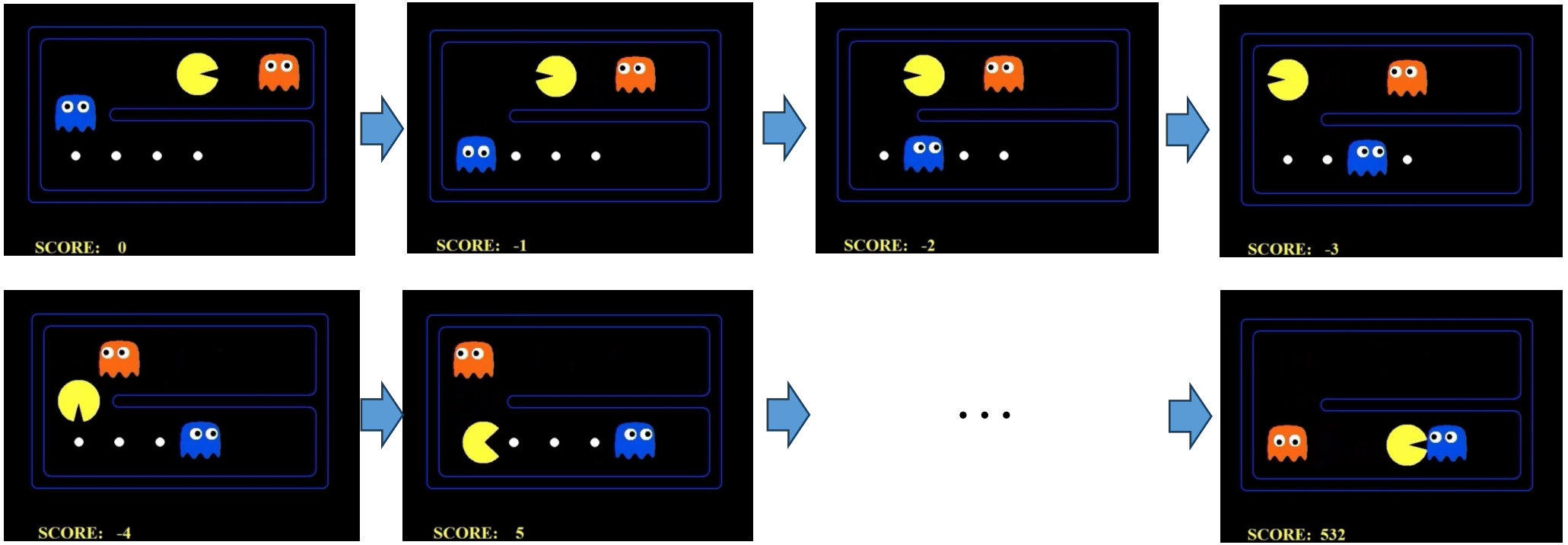
- **Game rules**:
  - Each move reduces 1 score.
  - Eating each dot will get 10 scores.
  - Being captured by the opponent will lose 500 scores and game ends (lose).
  - Eating all dots will get 500 scores and game ends (win).
  - The two opponents make decision independently

Perfect Opponent



- If the opponent always take the optimal action, the best choice is to move towards the opponent, to avoid producing more moves.

Random Opponent

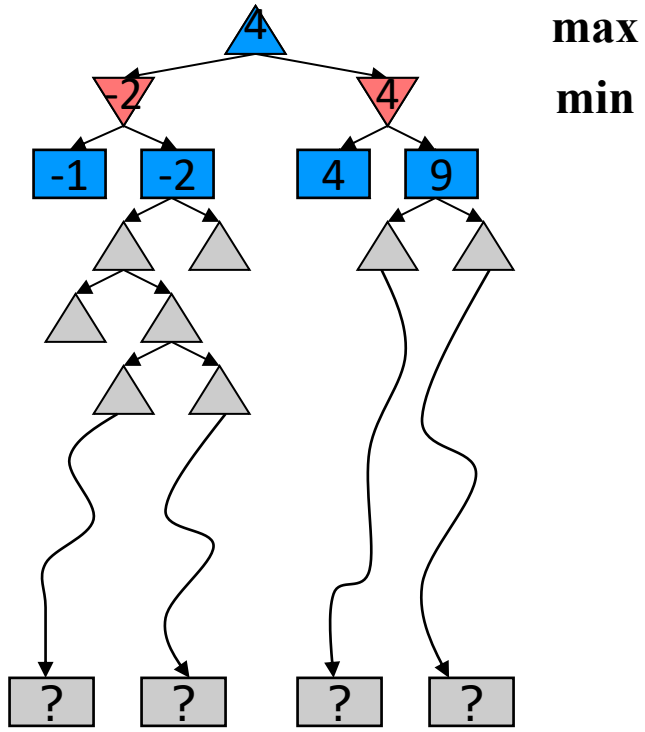


- If the opponent is random, then moving to the left may have a higher expected gain (chance to win)
- However, if Pacman still uses minimax algorithm, it will loss the chance to win.

Problem of Minimax - Resource Limits

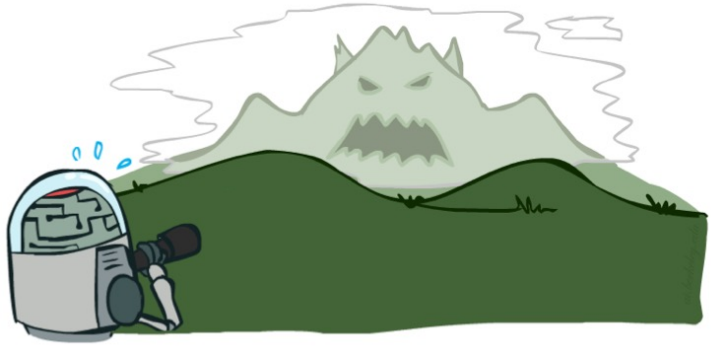
- In realistic games, cannot search to leaves!
- **Example:**
  - Suppose we have 100 seconds, can explore 10K nodes / sec
  - So can check 1M nodes per move, approx. depth of 8 with pruning – decent chess program

- **Solution:** Depth-limited search
  - Instead, search only to a limited depth in the tree
  - Replace terminal utilities with an **evaluation function** for non-terminal positions
  - **Guarantee of optimal play is gone**



Depth

- Evaluation functions are always imperfect
- The deeper in the tree the evaluation function is buried, the less the quality of the evaluation function matters
- An important example of the tradeoff between complexity of features and complexity of computation



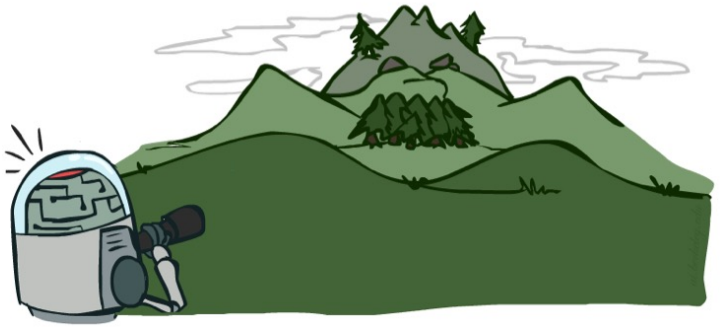
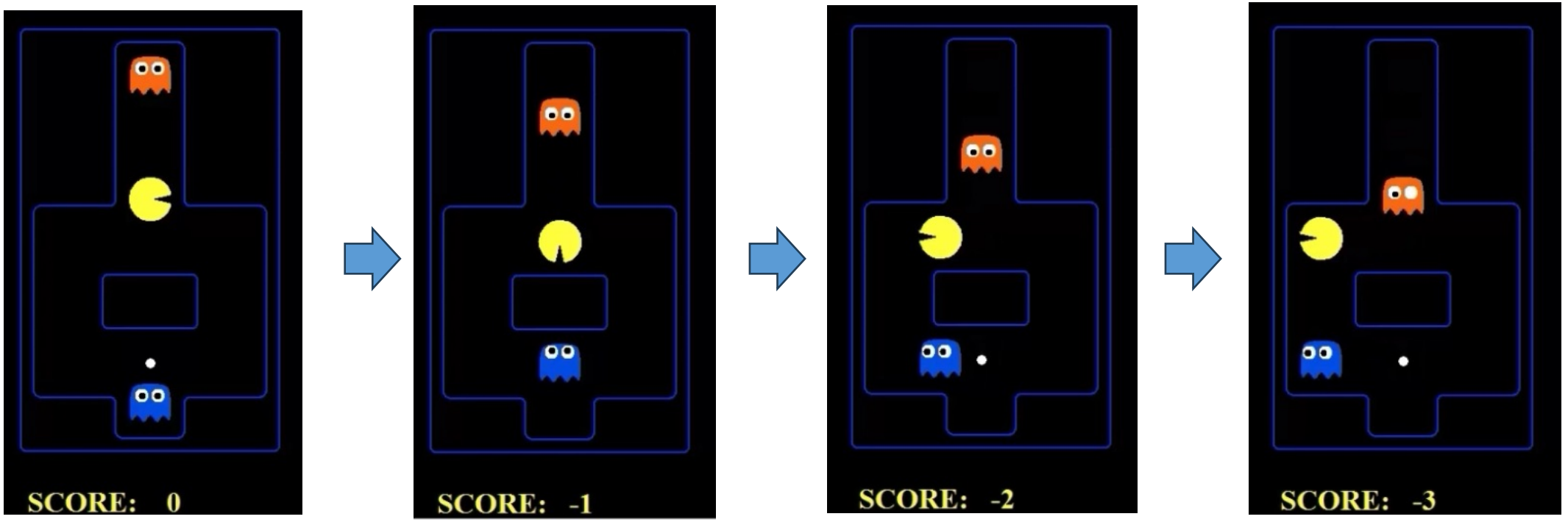
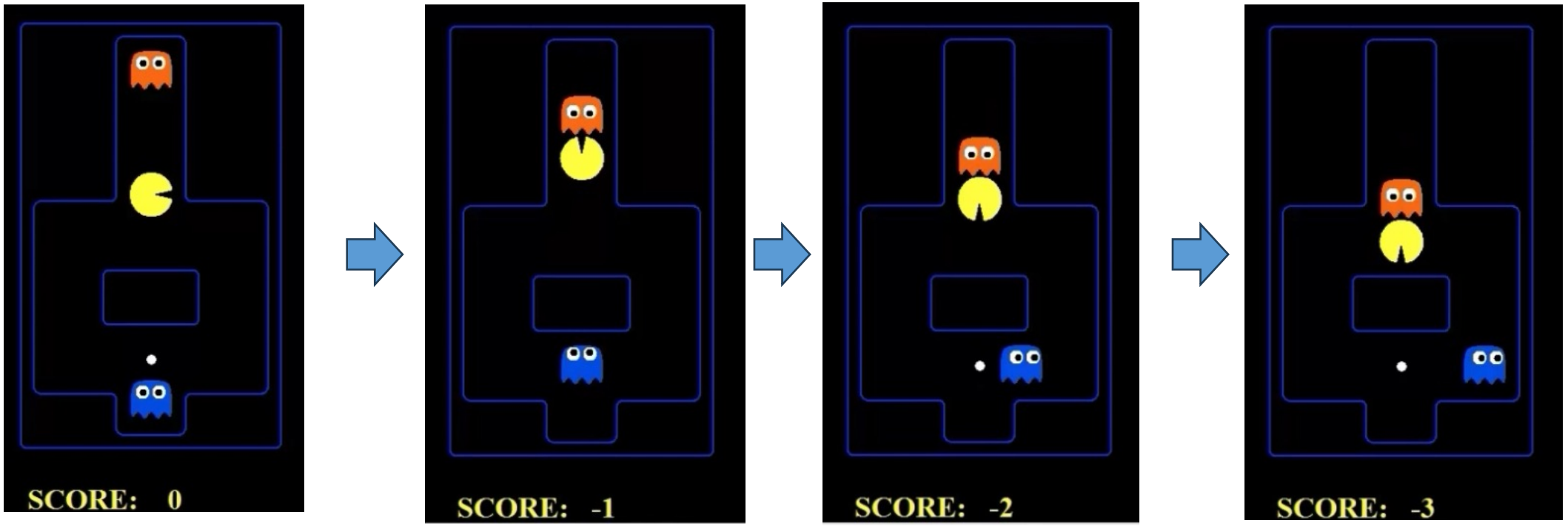


Illustration of Depth = 2



- If pacman only looks ahead for 2 steps (depth=2), moving down is safe.
- For the second move, moving left or right is safe, while moving top will lose the game
- Then, it will get surrounded by the two opponents.

Illustration of Depth = 10



- If pacman only looks ahead for 10 steps (depth=10), moving top can force the blue opponent to make decision and then act accordingly.
- After the blue opponent moves right, pacman can move left to avoid the opponent!

Cutting off search

MinimaxCutoff is identical to *MinimaxValue* except:

1. *Terminal?* is replaced by *Cutoff?*
2. *Utility* is replaced by evaluation function *Eval*

Does it work in practice?

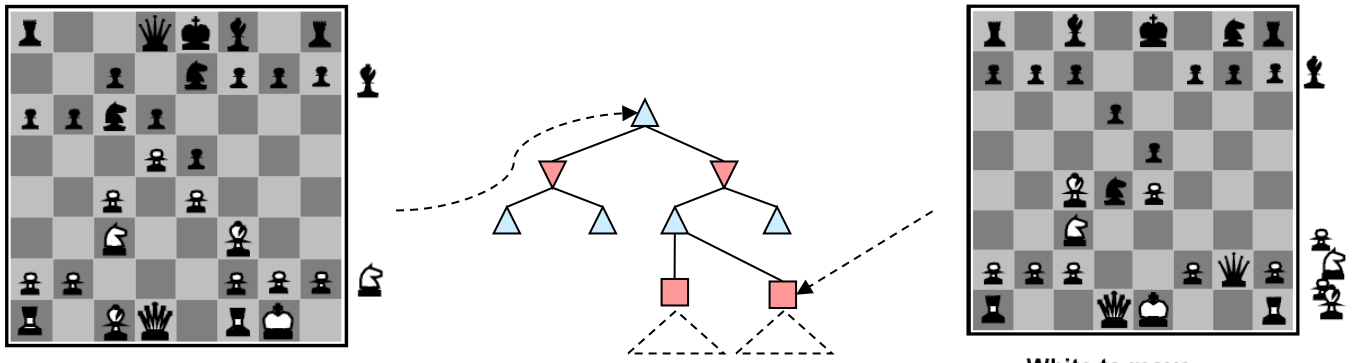
$b^m = 10^6$, $b = 35$ → $m = 4$

4-ply lookahead is a hopeless chess player!

- 4-ply ≈ human novice
- 8-ply ≈ typical PC, human master
- 12-ply ≈ Deep Blue, Kasparo

Evaluation Functions

- Evaluation functions score non-terminals in depth-limited search



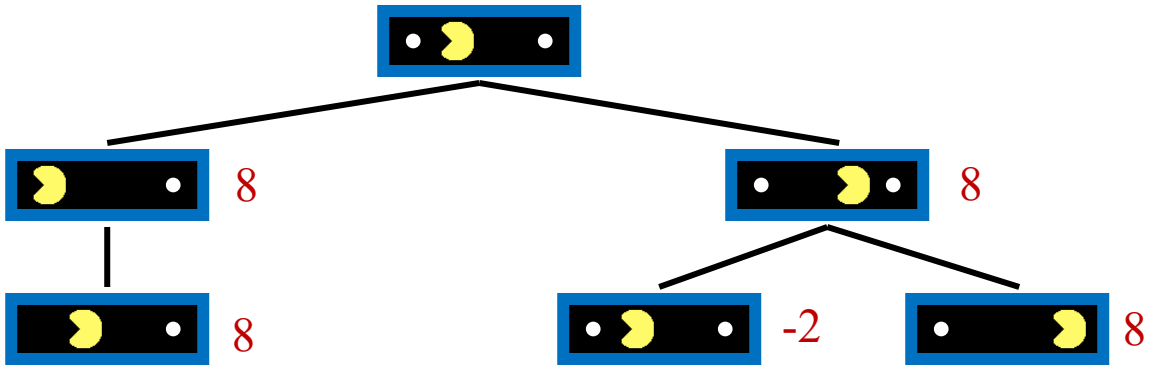
- Ideal function: returns the actual minimax value of the position
- In practice: typically weighted linear sum of features:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

- e.g. $f_1(s)$ = (num white queens – num black queens), etc.

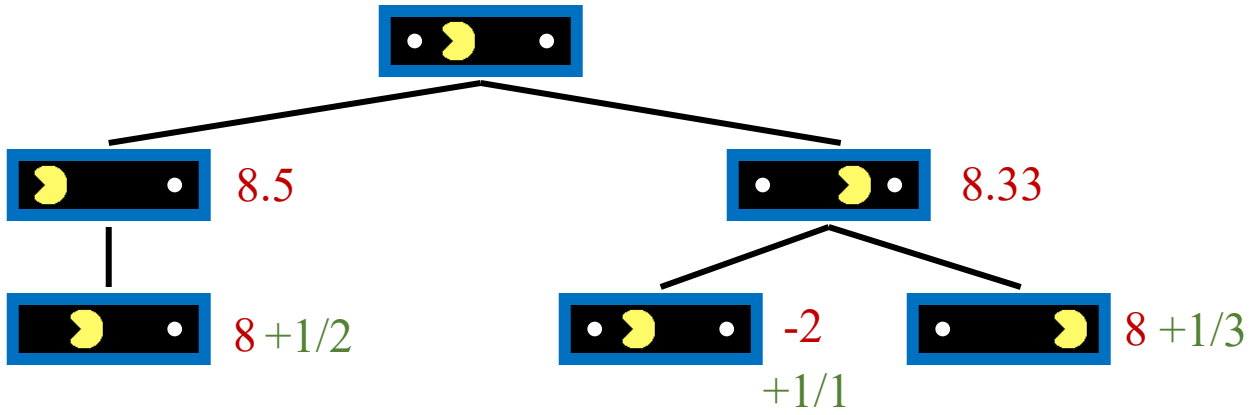
Problems of Bad Evaluation Functions

- Consider a depth 2 search for the pacman world below
  - Suppose we use the score at the state as evaluation function
  - The root node can go left or right
  - However, the directions are of the same evaluation value!
  - If you happen to break tie by choosing the state where the agent is close to a dot (the right branch), it reaches the same situation that both direction are of the same evaluation
  - The pacman will move back-and-forth – an agent keeps replanning.

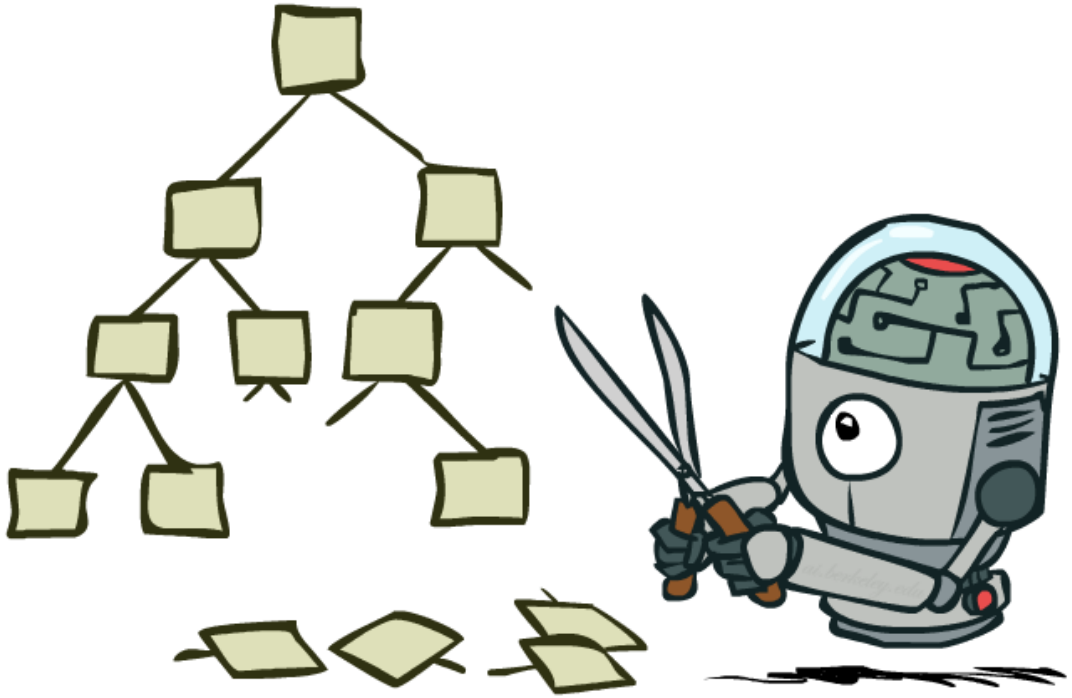


Problems of Bad Evaluation Functions

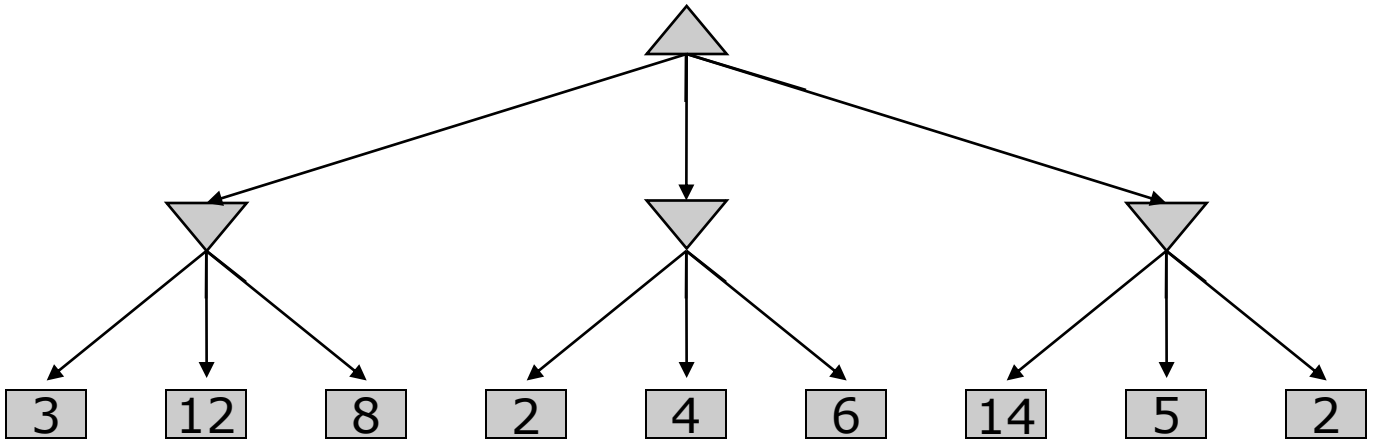
- This can be fixed by designing a better evaluation function
  - E.g., consider the distance to the nearest dot



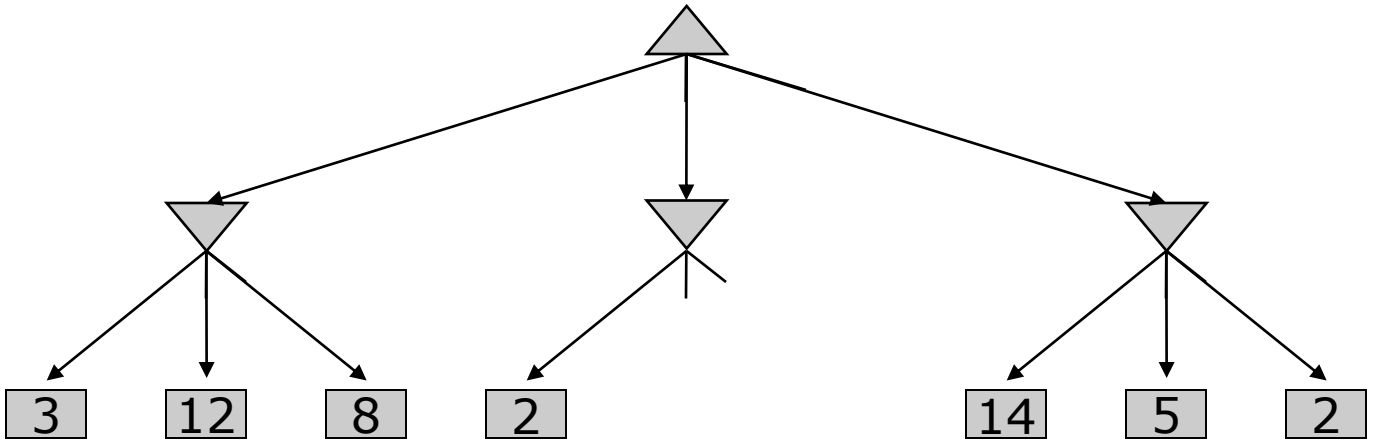
Game Tree Pruning



Minimax Example

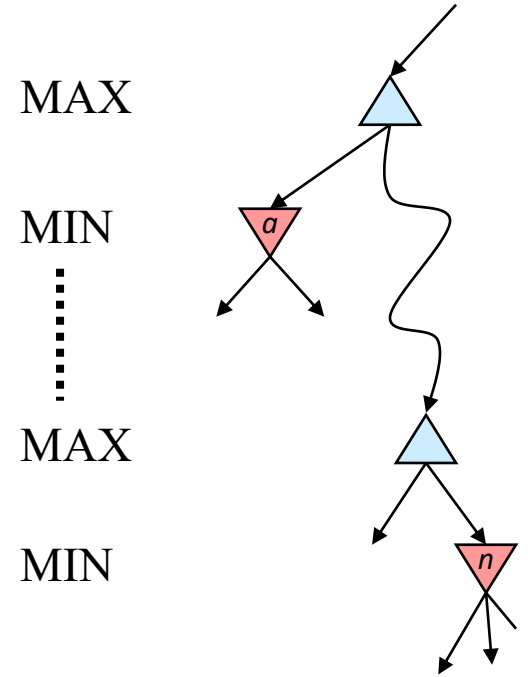


Minimax Pruning



Alpha-Beta Pruning

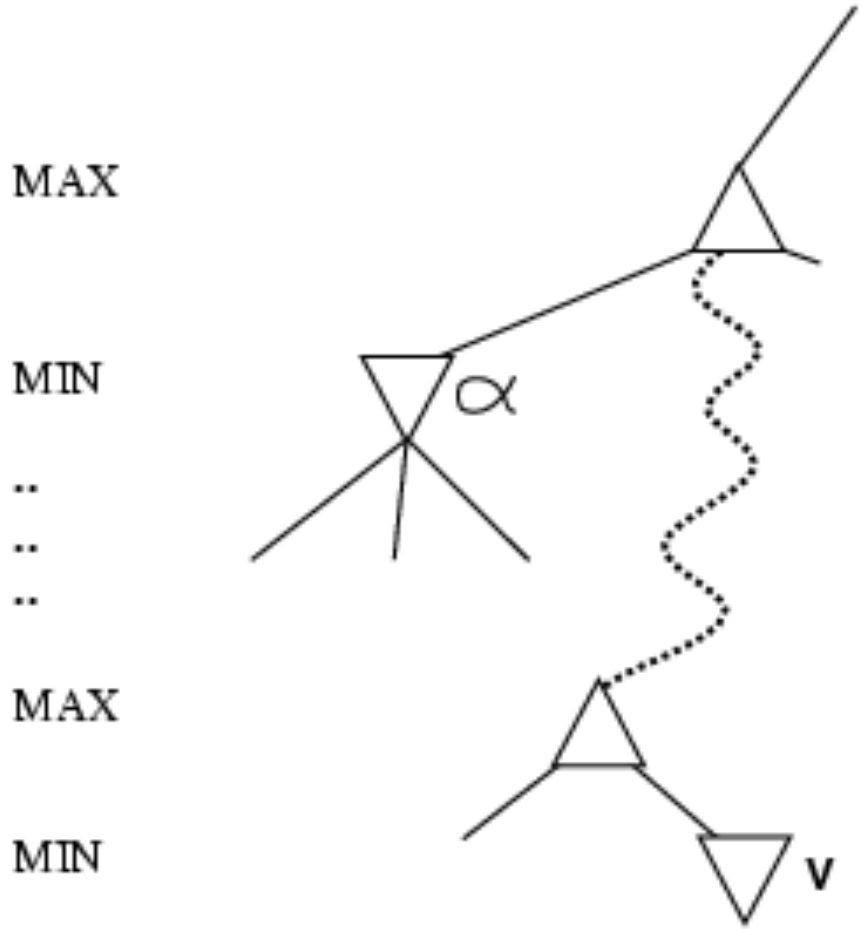
- General configuration (MIN version)
  - We're computing the MIN-VALUE at some node *n*
  - We're looping over *n*'s children
  - *n*'s estimate of the childrens' min is dropping
  - Who cares about *n*'s value? MAX
  - Let *a* be the best value that MAX can get at any choice point along the current path from the root
  - If *n* becomes worse than *a*, MAX will avoid it, so we can stop considering *n*'s other children (it's already bad enough that it won't be played)



- MAX version is symmetric

Why is it called α-β?

- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
- If *v* is worse than α, *max* will avoid it
  - prune that branch
- Define β similarly for *min*



The α-β algorithm

function ALPHA-BETA-SEARCH(*state*) **returns** *an action*
  inputs: *state*, current state in game
  $v \leftarrow$ MAX-VALUE($state, -\infty, +\infty$)
  return the *action* in SUCCESSORS(*state*) with value v

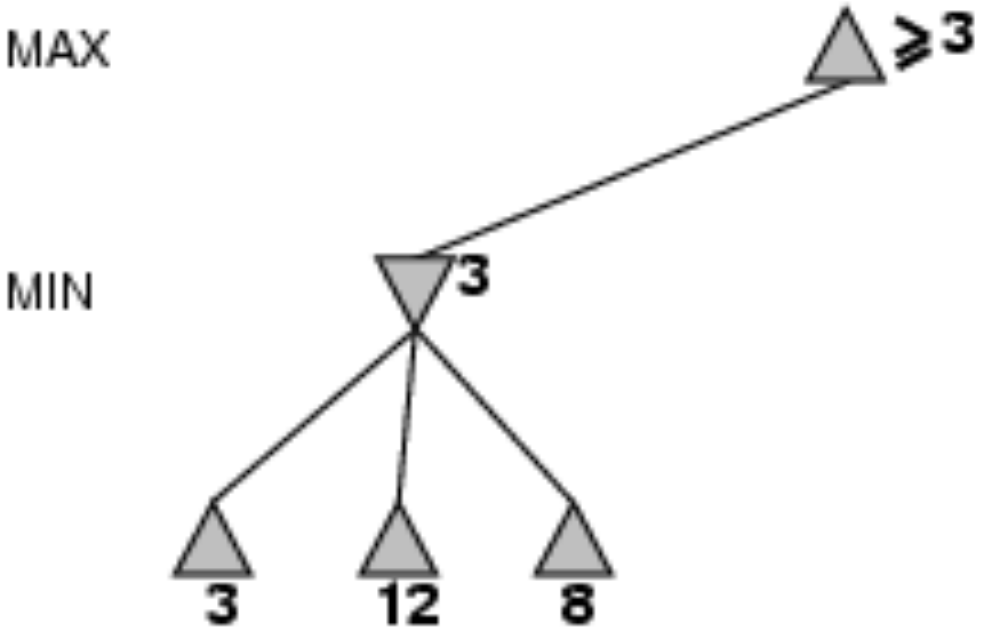
---

function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  inputs: *state*, current state in game
    α, the value of the best alternative for MAX along the path to *state*
    β, the value of the best alternative for MIN along the path to *state*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow -\infty$
  for a, s in SUCCESSORS(*state*) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(s, α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

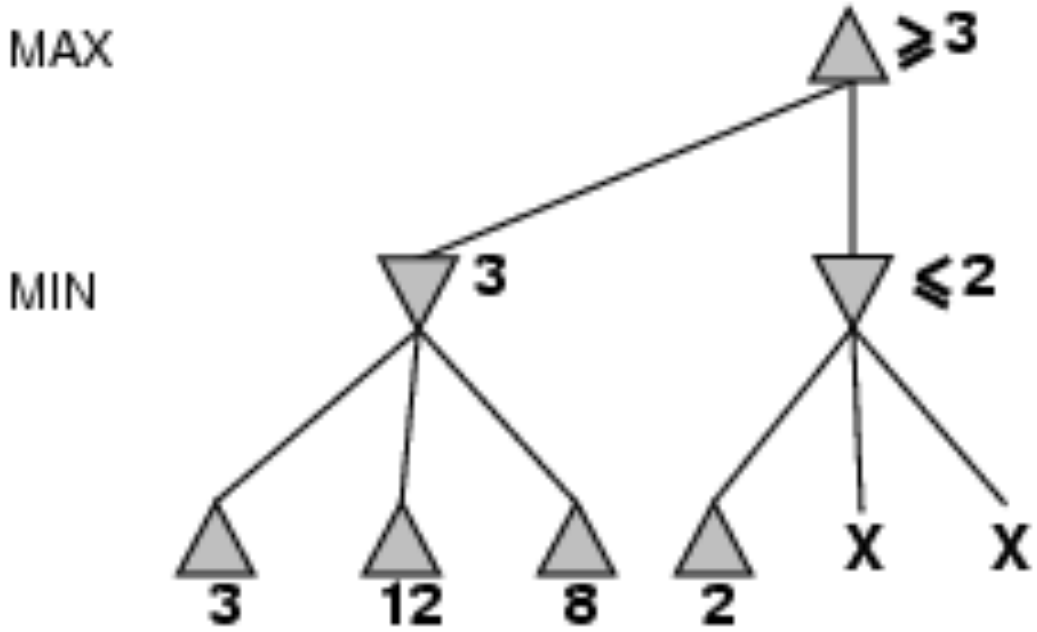
The α-β algorithm

function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  inputs: *state*, current state in game
    α, the value of the best alternative for MAX along the path to *state*
    β, the value of the best alternative for MIN along the path to *state*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow +\infty$
  for a, s in SUCCESSORS(*state*) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(s, α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

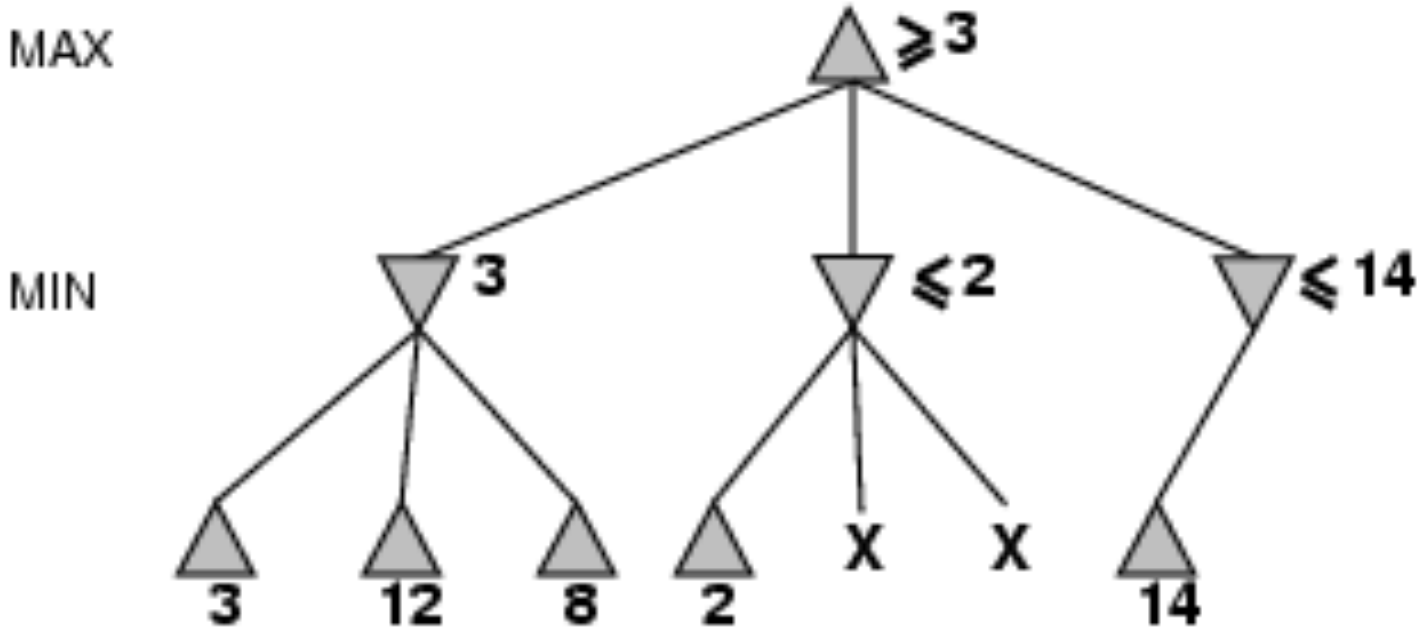
α-β pruning example



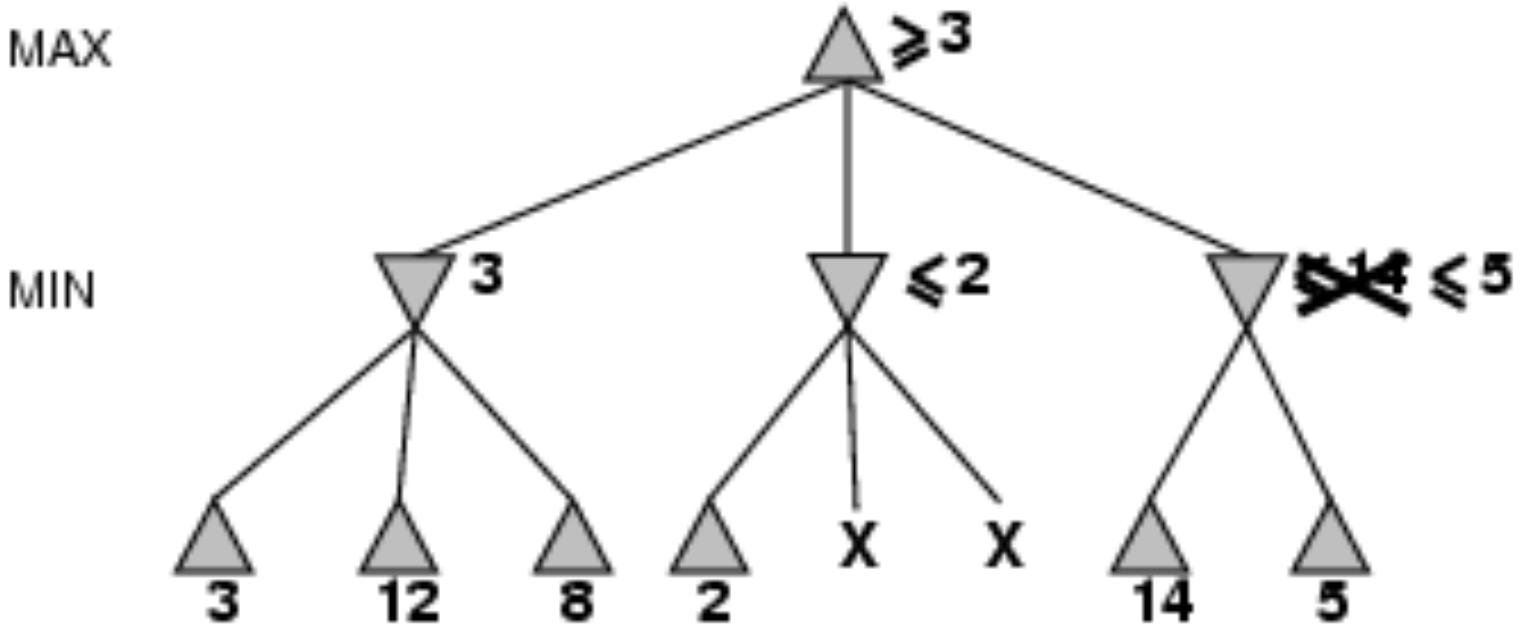
α-β pruning example



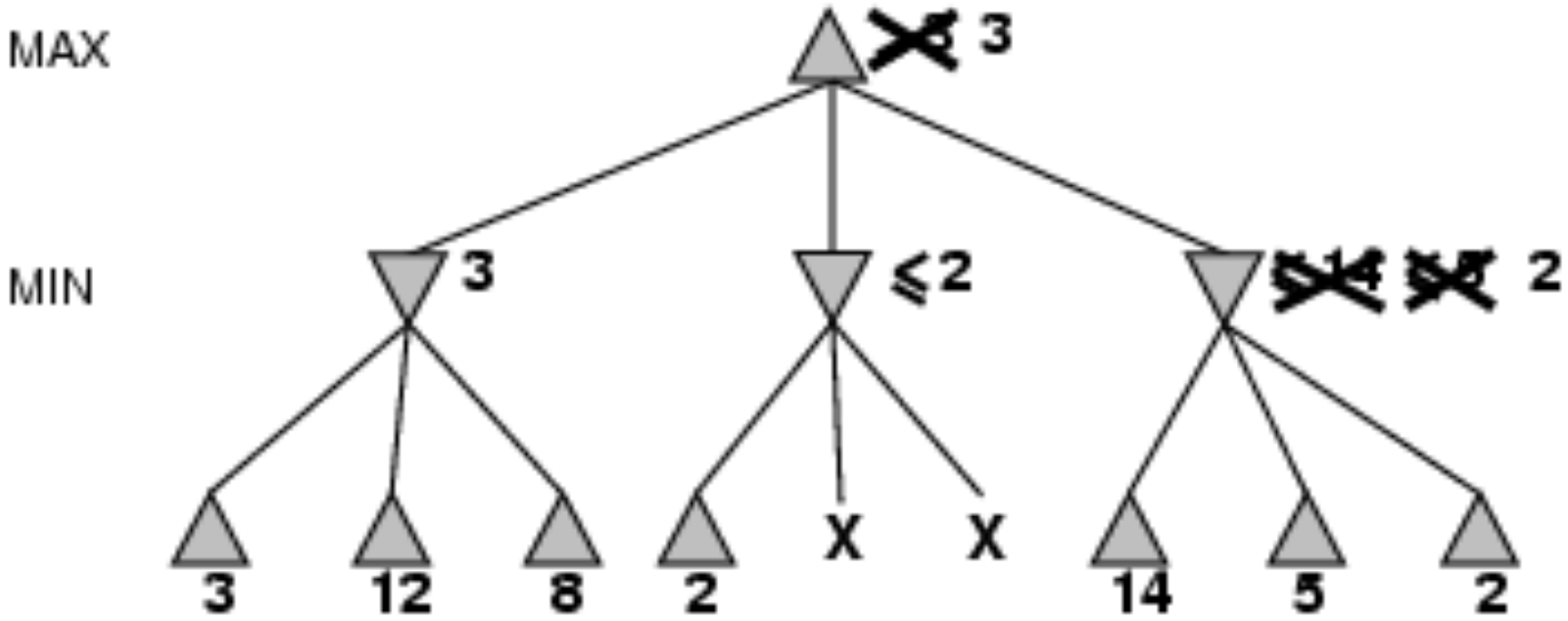
α-β pruning example



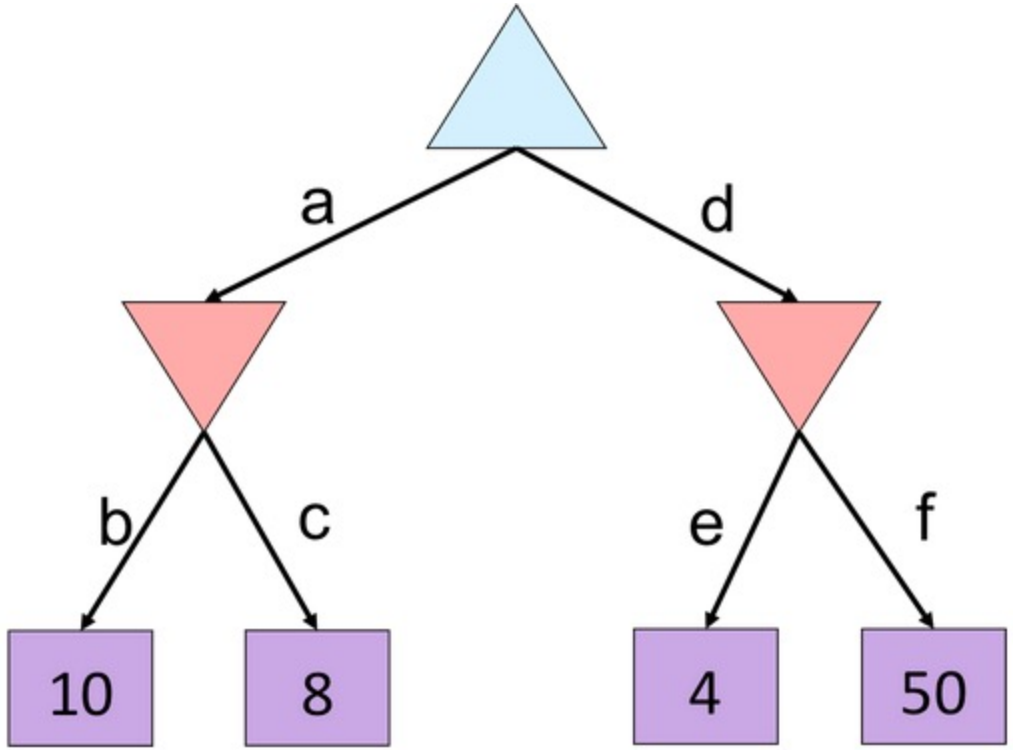
α-β pruning example



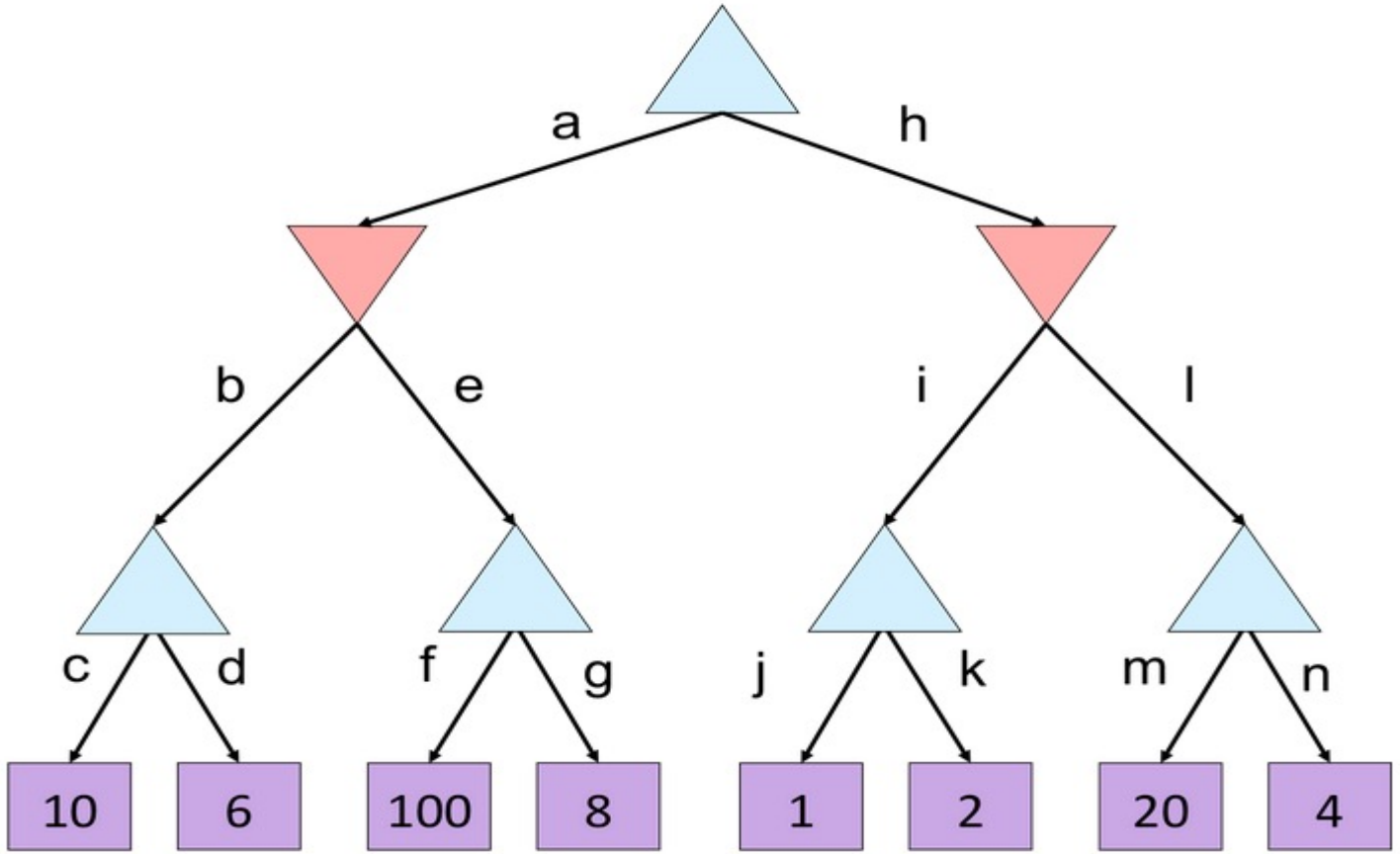
α-β pruning example



α-β Exercise



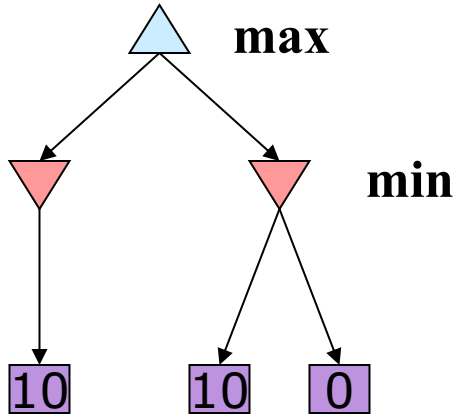
α-β Exercise



α-β pruning properties

- This pruning has no effect on minimax value computed for the root!

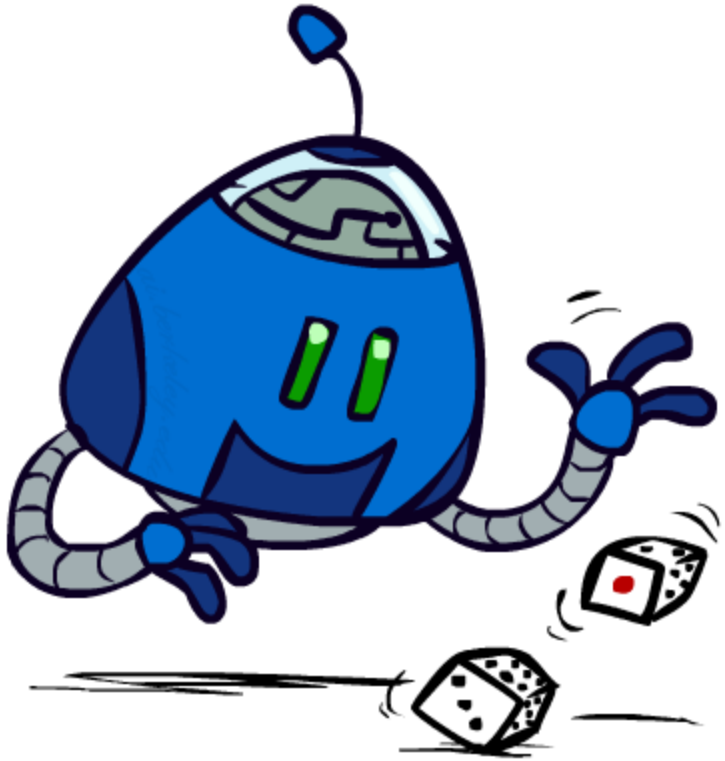
- Values of intermediate nodes might be very inaccurate
  - Important: children of the root may have very loose bounds
  - So the most naïve version won't let you do action selection



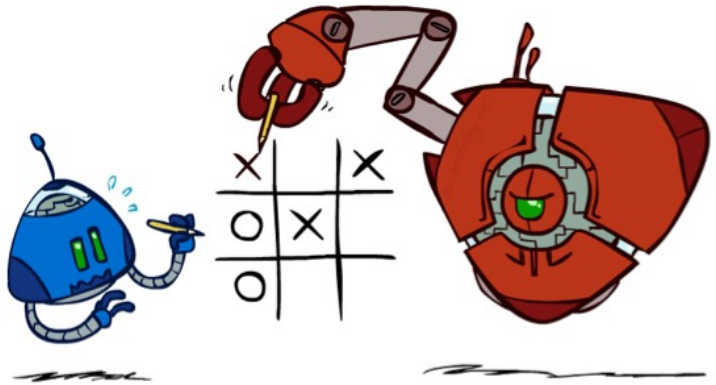
- Good child ordering improves effectiveness of pruning

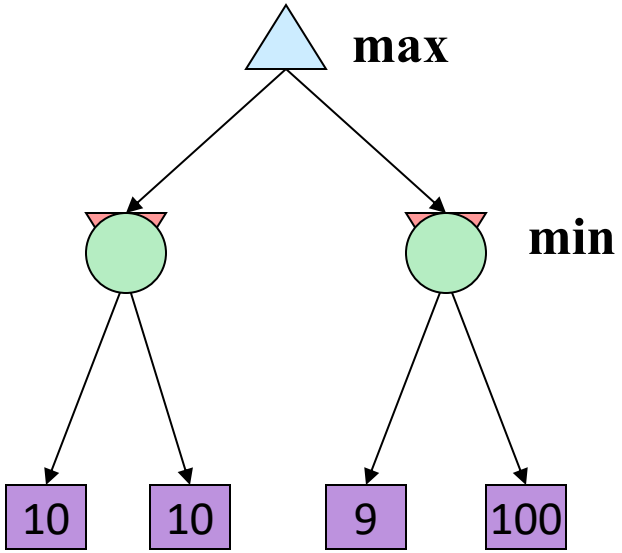
- With "perfect ordering":
  - Time complexity drops to $O(b^{m/2})$ (The effective branching factor becomes $\sqrt{b}$ instead of b, for chess about 6 instead of 35)
  - Doubles solvable depth!
  - Full search of, e.g. chess, is still hopeless…

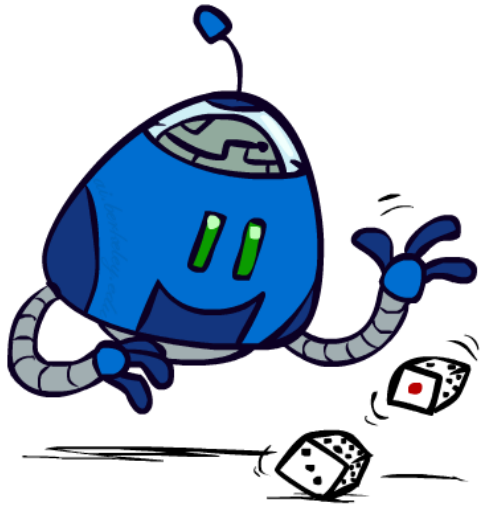
Uncertain Outcomes



Worst-Case vs. Average Case



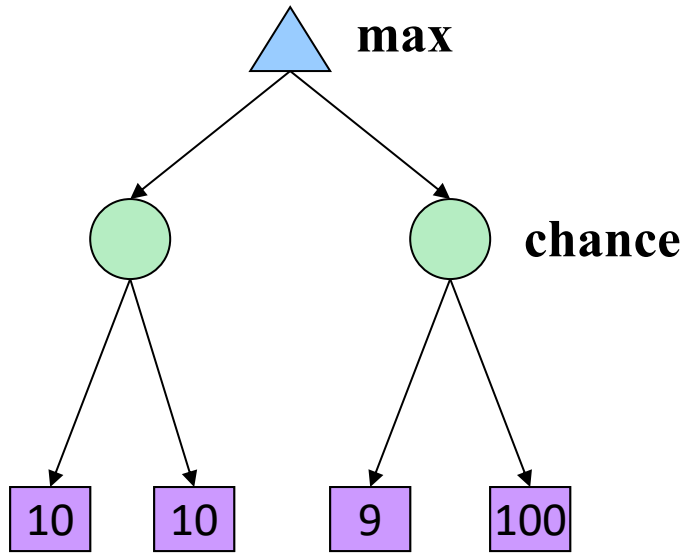




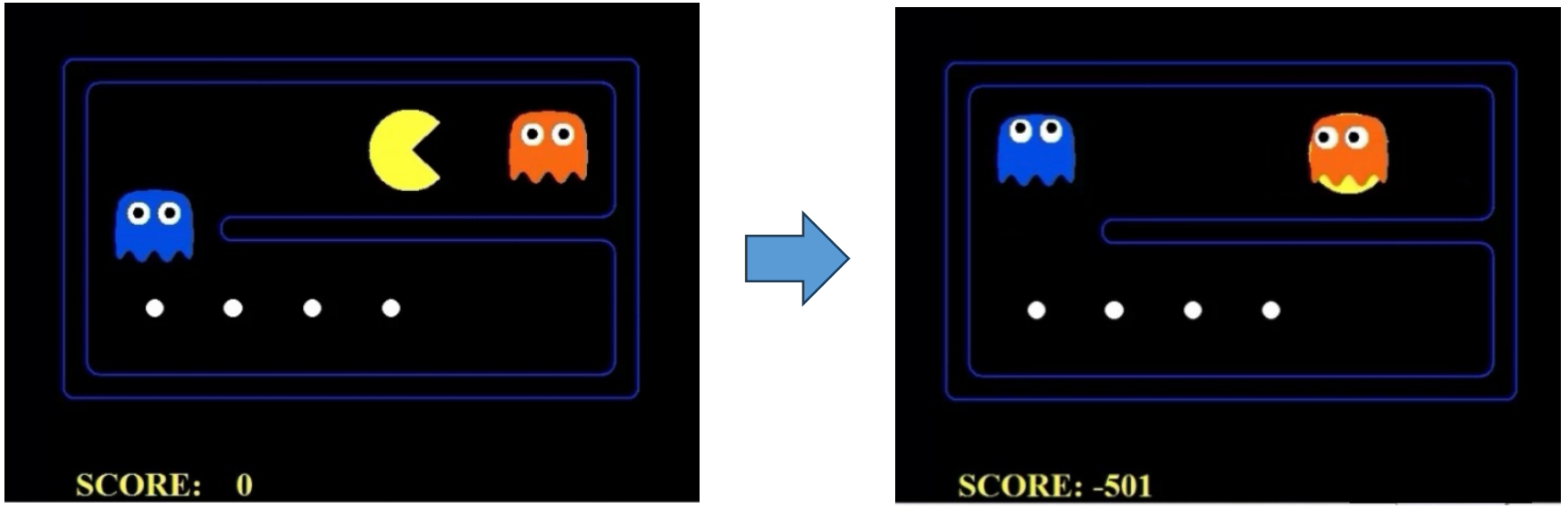
Idea: Uncertain outcomes controlled by chance, not an adversary!

Expectimax Search

- Why wouldn't we know what the result of an action will be?
  - Explicit randomness: rolling dice
  - Unpredictable opponents
  - Actions can fail: when moving a robot, wheels might slip
- Values should now reflect average-case (expectimax) outcomes, not worst-case (minimax) outcomes
- Expectimax search: compute the average score under optimal play
  - Max nodes as in minimax search
  - Chance nodes are like min nodes but the outcome is uncertain
  - Calculate their expected utilities

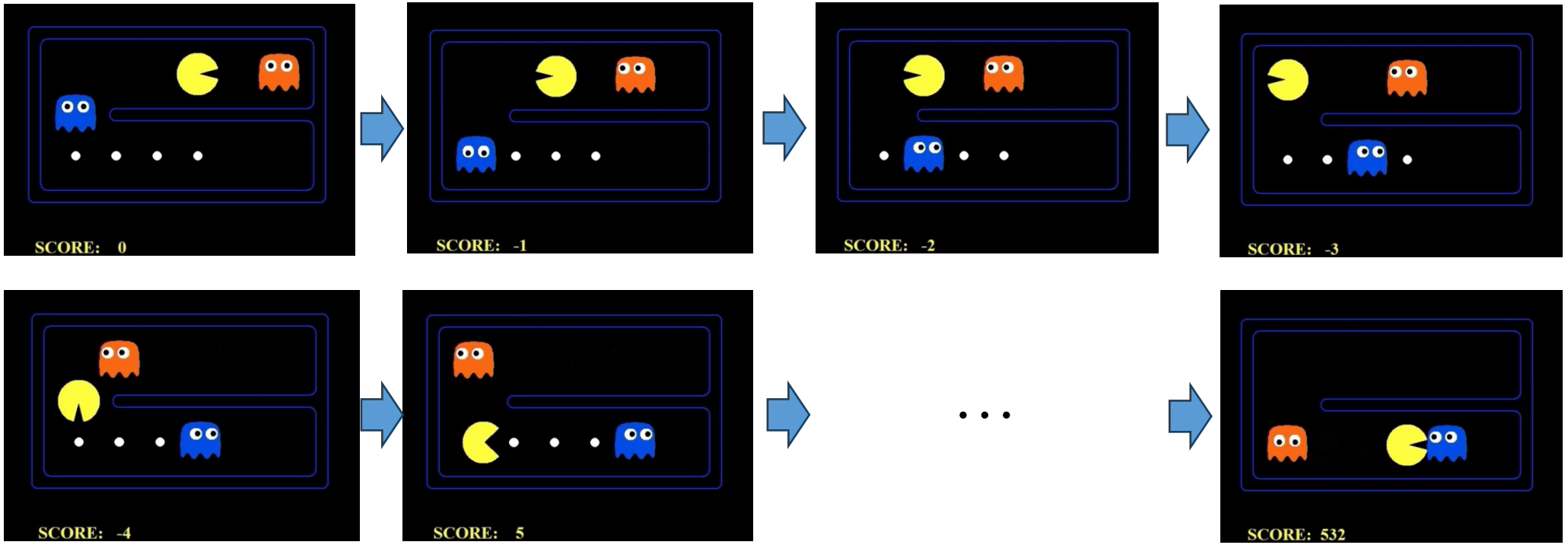


Minimax



- Assume the opponent always take the optimal action, it will move towards the opponent

Expectimax



- Assume the opponent performs some random actions. Moving to the left may create a higher expected value
- Note that pacman may still lose, when the blue opponent happens to go towards him.

Expectimax Pseudocode

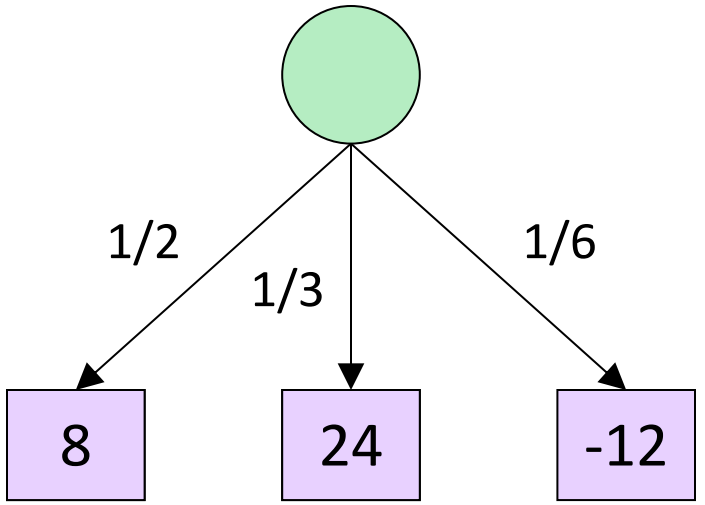
```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is EXP: return exp-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def exp-value(state):
    initialize v = 0
    for each successor of state:
        p = probability(successor)
        v += p * value(successor)
    return v
```

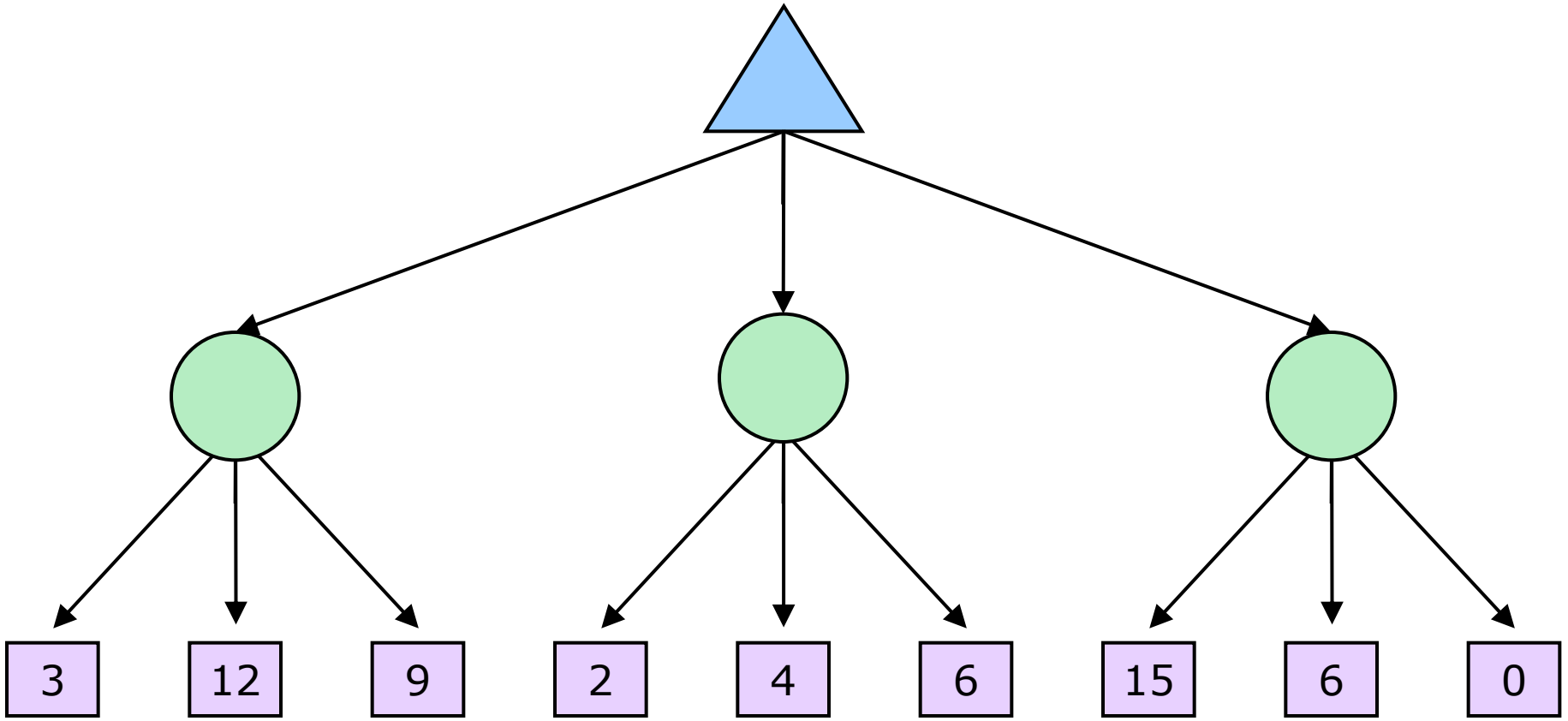
Expectimax Pseudocode

```
def exp-value(state):
    initialize v = 0
    for each successor of state:
        p = probability(successor)
        v += p * value(successor)
    return v
```

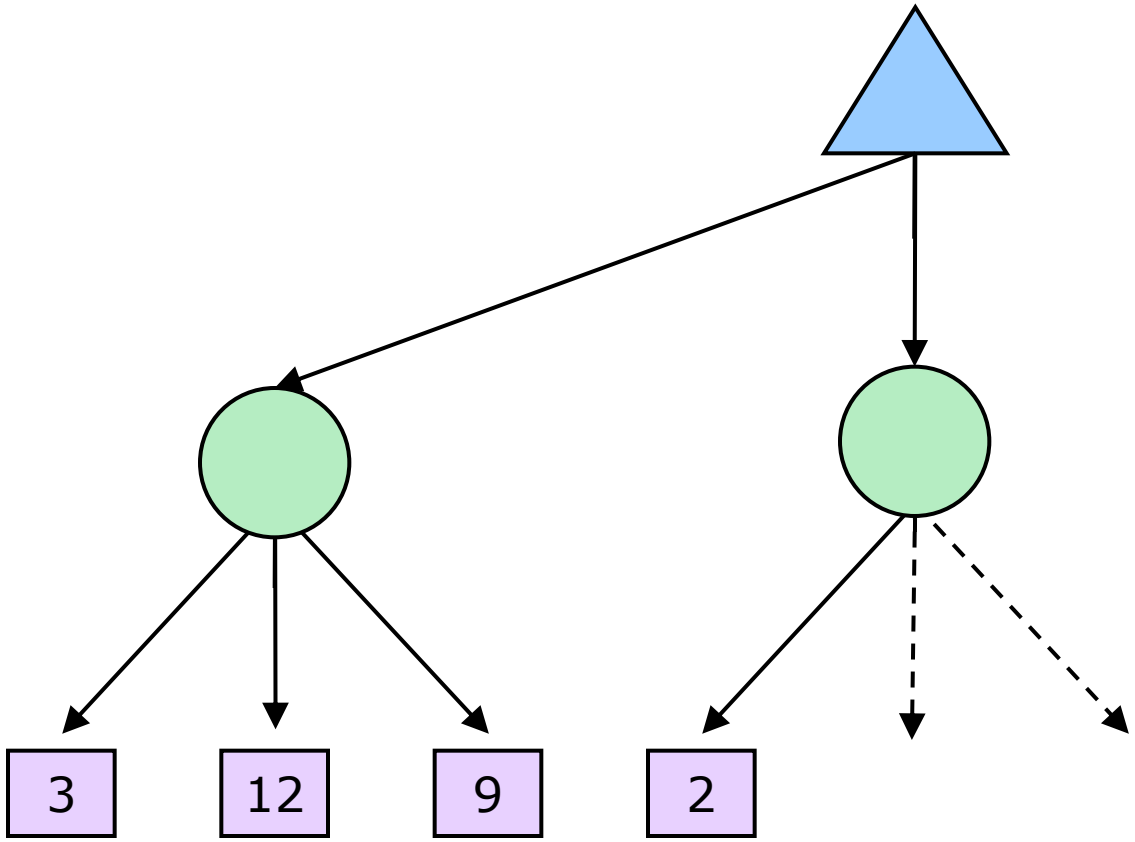


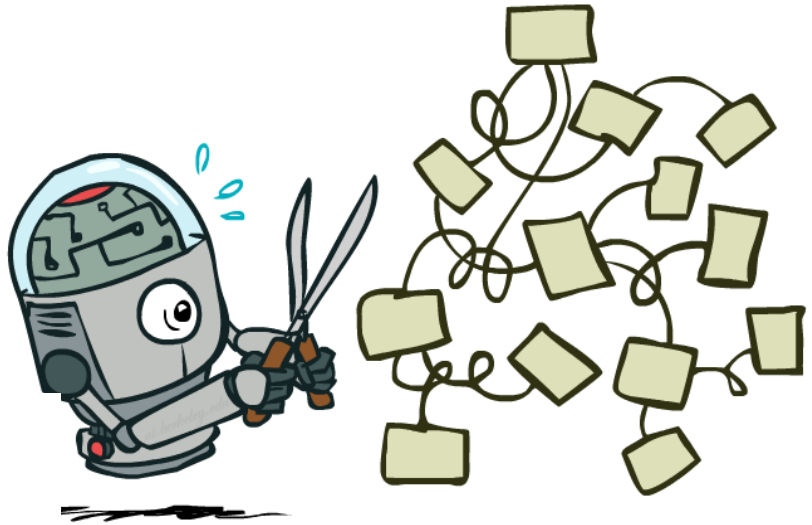
$$v = (1/2)\ (8) + (1/3)\ (24) + (1/6)\ (-12) = 10$$

Expectimax Example

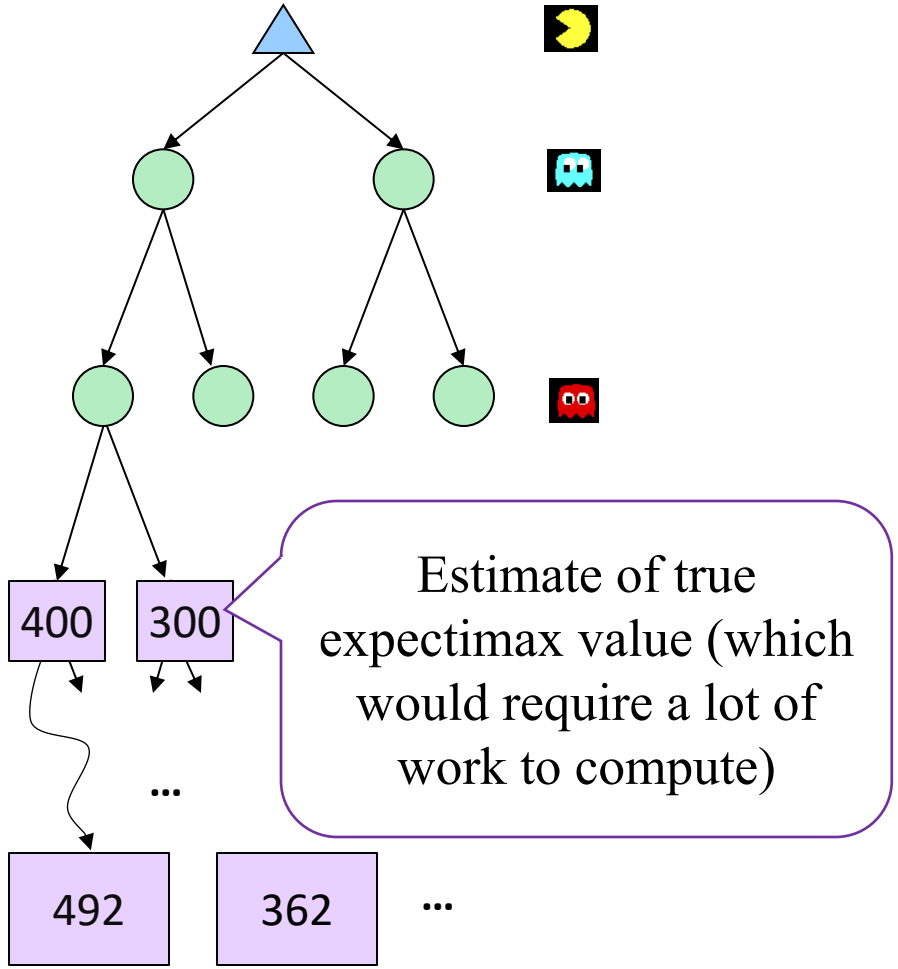


Can we perform pruning in expectimax algorithm?

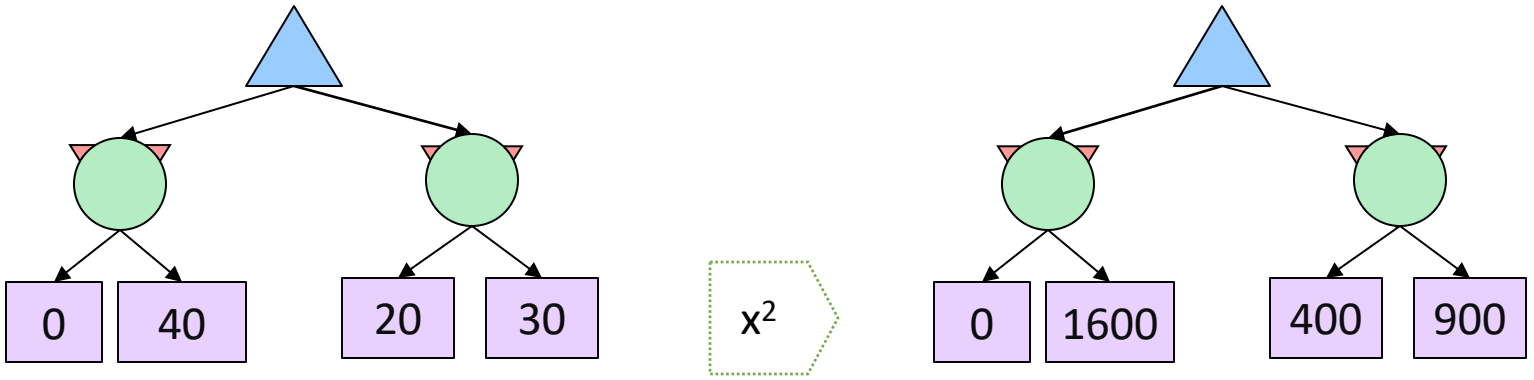




Depth-Limited Expectimax



What Utilities to Use?



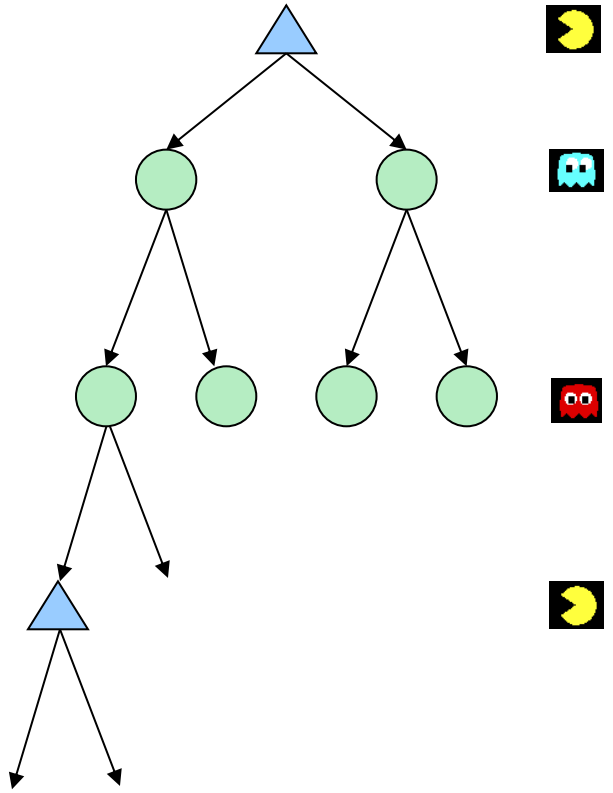
- For worst-case minimax reasoning, terminal function scale doesn't matter
  - We just want better states to have higher evaluations (get the ordering right)
  - We call this insensitivity to monotonic transformations

- For average-case expectimax reasoning, we need *magnitudes* to be meaningful

What Probabilities to Use?

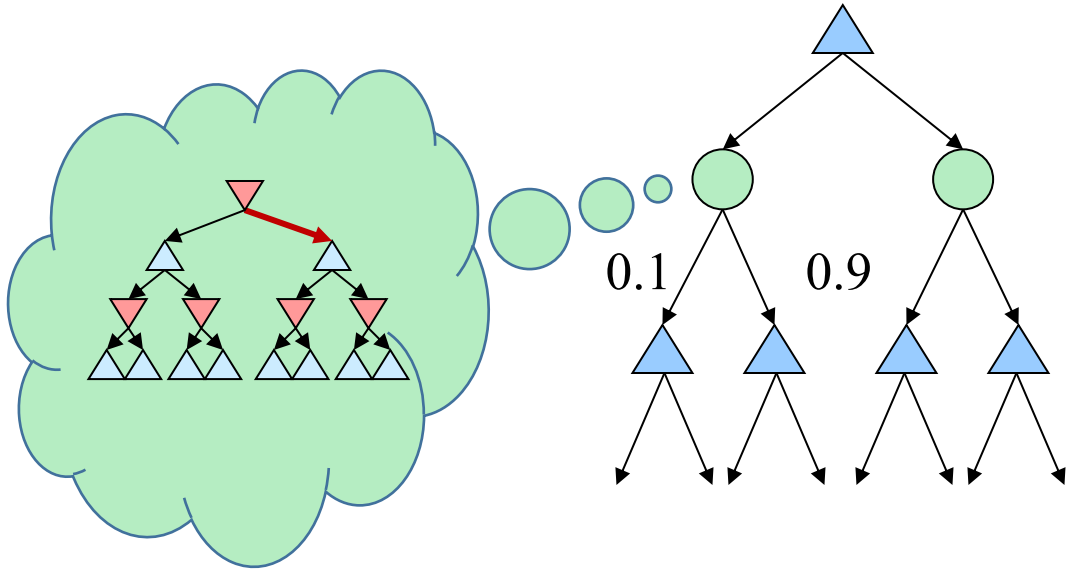
- In expectimax search, we have a probabilistic model of how the opponent will behave in any state
  - Model could be a simple uniform distribution (roll a die)
  - Model could be sophisticated and require a great deal of computation

- In this lecture, we assume each chance node magically comes along with probabilities that specify the distribution over its outcomes



Example: Informed Probabilities

- Let's say you know that your opponent is actually running a depth 2 minimax, using the result 80% of the time, and moving randomly otherwise
- Question: What tree search should you use?



- Answer: Expectimax!
  - To figure out EACH chance node's probabilities, you have to run a simulation of your opponent
  - This gets very slow very quickly
  - Even worse if you have to simulate your opponent simulating you…

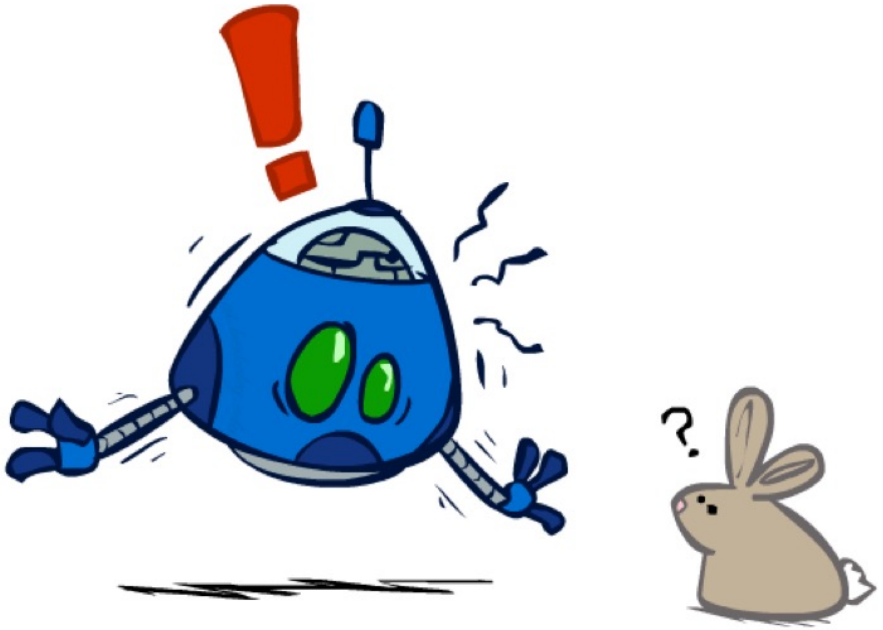
The Dangers of Optimism and Pessimism

Dangerous Optimism

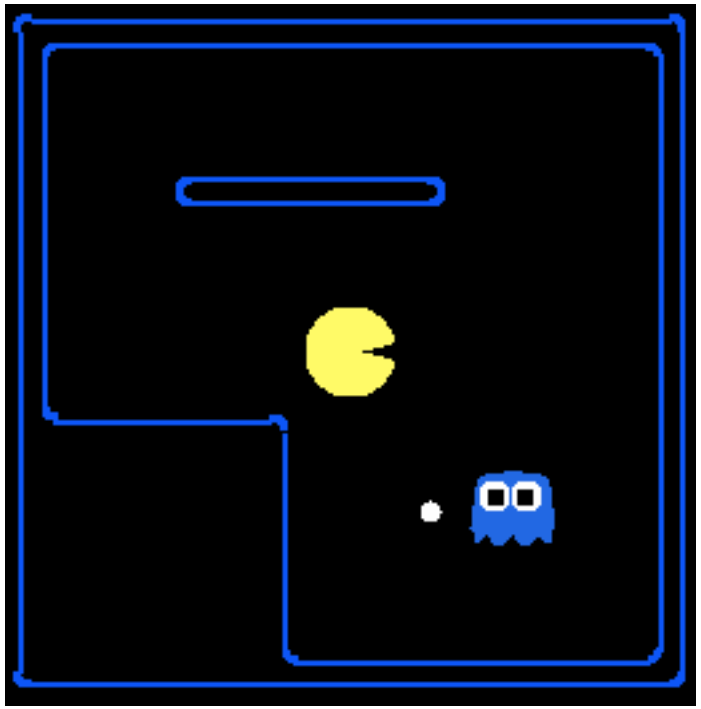
Assuming chance when the world is adversarial

Dangerous Pessimism

Assuming the worst case when it's not likely



Assumptions vs. Reality



	Adversarial Ghost	Random Ghost
Minimax Pacman	Won 5/5 Avg. Score: 483	Won 5/5 Avg. Score: 493
Expectimax Pacman	Won 1/5 Avg. Score: -303	Won 5/5 Avg. Score: 503

Results from playing 5 games

Pacman used depth 4 search with an eval function that avoids trouble
Ghost used depth 2 search with an eval function that seeks Pacman

Summary

The following are the main knowledge points covered:

- Zero-sum game and game tree
- Adversarial search
  - Minimax algorithm
  - Alpha-Beta Pruning algorithm
- Uncertainty
  - Expectimax algorithm
  - Exploration v.s. exploitation